

Evaluation and Practical Use of the ReFinED Entity Linking Tool

Benchmarking Against SemTab Datasets and System Integration

Børge Bjørnstadjordet

Informatics: Programming and Systems Architecture
60 ECTS study points

Department of Informatics
Faculty of Mathematics and Natural Sciences

Børge Bjørnstadjordet

Evaluation and Practical Use of the ReFinED Entity Linking Tool

Benchmarking Against SemTab Datasets and
System Integration

Supervisors:

Dumitru Roman, SINTEF

Roberto Avogadro, SINTEF

Professor Martin Giese, University of Oslo

Abstract

This thesis evaluates the ReFinED entity linking system by benchmarking its performance on tabular data against other existing state-of-the-art Semantic Table Interpretation (STI) approaches. Focusing specifically on Cell Entity Annotation (CEA), this study assesses ReFinED’s zero-shot-capable neural approach across a range of benchmark datasets, examining how row-level contextual information affects disambiguation and how supervised fine-tuning on domain-specific data can improve performance over the general-purpose pre-trained baseline model. Beyond performance metrics, this research addresses the practical challenge of deploying large-scale transformer models as web services. A system architecture featuring a FastAPI back-end is proposed and implemented, integrating ReFinED into the Koala-UI reconciliation platform. This demonstrates how resource-intensive entity linking models can be transitioned from research benchmarks into functional web services. The results provide a multi-faceted perspective on ReFinED: its algorithmic accuracy on tabular data, the impact of domain adaptation through fine-tuning, and the technical trade-offs required for practical deployment.

Declaration of the Use of Artificial Intelligence

Generative artificial intelligence tools were used as aids in selected parts of the writing and development process. All content, claims, and references are made by the author, and all AI-assisted contributions have been thoroughly reviewed, edited, and verified before inclusion.

Claude Sonnet 4.6, developed by Anthropic, was primarily used for code-related assistance, including support in developing analysis scripts, curve-generations, and Docker configurations. It was also used to a lesser extent for linguistic and stylistic changes of written text and for structural guidance during outlining of certain sections and chapters. The built-in AI assistance provided by Overleaf was also used for grammar and phrasing corrections.

Acknowledgement

I would like to thank my supervisors Dumitru Roman, Roberto Avogadro, and Professor Martin Giese for their roles in making this thesis possible. In particular, I want to thank Roberto for his consistent guidance and support throughout the process, for our regular meetings, and for his practical help with the technical side of the work, including Docker configuration and the deployment of the ReFinED API service demonstration.

I would also like to thank my friends and family for their encouragement and support along the way. Their support throughout what has been a long and demanding process has been greatly appreciated.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Goals of the Thesis	2
1.3	Research Questions	3
1.4	Statement of Contributions	4
1.5	Thesis Structure	5
2	Theoretical Background and Related Work	6
2.1	Semantic Technologies and Knowledge Graphs	6
2.1.1	Ontologies and Knowledge Graphs	7
2.1.2	The Semantic Technology Stack	7
2.2	Entity Linking: Concepts and Challenges	8
2.2.1	Surface Forms and Ambiguity	9
2.2.2	Transformers and Embeddings	10
2.3	Entity Linking for Tabular Data	11
2.3.1	Structural Context and Data Quality	12
2.4	Approaches to Tabular Entity Linking	12
2.5	The SemTab Challenge	16
2.5.1	Challenge Structure	16
2.5.2	Evaluation Metrics and Trade-offs	17
2.5.3	Relevance to this Thesis	18
3	Datasets	19
3.1	SemTab Selection and Data Characteristics	19
3.1.1	Standard Web Tables	19
3.1.2	HTR1 and HTR2	20
3.1.3	HardTables and Adversarial Mechanisms	21
3.1.4	Domain-Specific Datasets	21
3.2	Dataset Selection and Scope	22
3.3	Structural Characteristics	23

3.3.1	Computational Demands and Annotation Density	24
3.3.2	Naming Variability and Ambiguity	25
3.4	Data Preparation and Preprocessing	26
3.5	Evaluation Metrics and NIL-Awareness	27
3.6	Limitations	28
4	Implementation and System Integration	29
4.1	Computational Environment and Infrastructure	29
4.2	Model Configuration and Inference Strategy	31
4.3	Automated Evaluation Pipeline	33
4.4	Back-end Service Architecture	35
4.4.1	REST API Design and Endpoints	35
4.4.2	Model Serving and Memory Management	36
4.5	Integration with Koala-UI	37
4.5.1	Visualisation of Linking Results	37
4.6	Practical Limitations	40
5	Evaluation	42
5.1	Experimental Design	43
5.1.1	Comparison Methodology	43
5.1.2	Experimental Hypotheses	44
5.2	Ablation Study	45
5.3	Results and Comparative Analysis	49
5.3.1	Precision-Recall Dynamics	49
5.3.2	Efficiency vs. Accuracy Trade-offs	52
5.3.3	Comparative Analysis	54
5.4	Qualitative Error Analysis	55
5.5	Evaluation Summary	57
6	Domain Adaptation and Optimisation	59
6.1	Technical Rationale for Fine-Tuning	59
6.2	Fine-Tuning Methodology and Configuration	60
6.3	Evaluation Strategy	61
6.4	Results: The Impact of Domain Specialisation	61
6.5	Summary and Implications	63
7	Discussion	65
7.1	Strengths and Limitations of ReFinED	65
7.2	Dataset-Specific Observations	67
7.3	Implications for Practical Deployment	68
7.3.1	Inference Mode Trade-offs	69
7.3.2	Domain Expert Architecture	69

7.3.3	Human-in-the-Loop Utility	70
8	Conclusion and Future Work	71
8.1	Conclusion	71
8.2	Future Work	72
A	Experimental Logs and Benchmark Data	79
A.1	Data Repository Access	79
A.2	Repository Structure and Contents	79
B	Dataset Repository References	81
C	API Endpoint Specifications	82
D	Source Code Snippets	83

List of Figures

2.1	Process of Entity Linking: The model utilises row-wise context (neighbouring cells) to resolve surface form ambiguity.	9
2.2	2D PCA projection of entity embeddings with categorical decision boundaries.	10
3.1	Row-wise context disambiguation: neighbouring cells (State and Landmark) resolve the ambiguous “Springfield” mention to the correct Wikidata QID.	25
4.1	Execution flow of the evaluation pipeline, from dataset loading through inference to metric computation and result logging.	33
4.2	Automated evaluation pipeline organised across input, processing, and output layers, showing script roles and data flow	34
4.3	Koala-UI settings page showing the reconciliation services panel with ReFinED registered as a provider alongside Crocodile, LionLinker, and the Wikidata reconciler.	38
4.4	Koala-UI automatic column type identification interface, where the moose module uses a configured LLM to assign NE and LIT subtypes to table columns prior to reconciliation.	38
4.5	Koala-UI reconciler selection interface showing ReFinED selected as the active provider prior to running entity linking on the dataset.	39
4.6	Koala-UI dataset view following entity linking with ReFinED, showing the top-3 results with candidate entity references per row.	39
4.7	Koala-UI candidate exploration panel for <i>The Shawshank Redemption</i> , showing top- k candidates returned by ReFinED with confidence scores, Wikidata QIDs, entity types, and the top-ranked prediction highlighted as winner. . .	40
5.1	PR curves and F1 vs. confidence threshold, specialised datasets, cell mode. SN (AUC=0.469) is notably weaker than companies (AUC=0.650) and movies (AUC=0.623), reflecting its high <i>NIL</i> ratio.	50

5.2	PR curves and F1 vs. confidence threshold, specialised datasets, row mode. <code>companies</code> (AUC=0.690) and <code>movies</code> (AUC=0.744) improve over cell mode; <code>SN</code> (AUC=0.482) remains constrained by its <i>NIL</i> ratio.	51
5.3	PR curves and F1 vs. confidence threshold, SemTab datasets, cell mode. <code>Round1_T2D</code> (AUC=0.735) leads, while <code>HardTablesR3</code> (AUC=0.284) re- fleets adversarial complexity.	51
5.4	PR curves and F1 vs. confidence threshold, SemTab datasets, row mode. <code>Round1_T2D</code> (AUC=0.773) improves over cell mode; <code>HardTablesR3</code> (AUC=0.262) degrades slightly, consistent with adversarial noise sensitivity.	52
6.1	Accuracy change relative to the stock baseline for each fine-tuned model across both domains. Negative values indicate degradation relative to the stock model.	62

List of Tables

3.1	Structural Characteristics of Evaluated SemTab Datasets.	23
3.2	Structural Characteristics of Specialised Evaluation Datasets.	24
4.1	Experimental Hardware Configurations	30
5.1	Ablation A vs B: Cell-only vs Row-context F1 (Stock Model)	46
5.2	Ablation C: Stock vs. Fine-tuned Model (F1 Score)	47
5.3	Ablation D: Cell-only vs. Column Header Label F1 (Stock Model)	48
5.4	System Performance Summary: Efficiency vs. Accuracy	49
5.5	Throughput and VRAM by Inference Mode (Stock Model on System B) . .	53
5.6	CEA F1 Comparison: ReFinED vs. STI Reference Systems	54
6.1	Fine-tuning Results: F1-score, Precision (P), and Recall (R) (Row Mode) .	62
B.1	Zenodo Record IDs and DOI Mappings for Evaluated Corpora.	81
C.1	ReFinED API Endpoint Summary	82

Chapter 1

Introduction

1.1 Background and Motivation

The rapid growth of digital data has created unprecedented opportunities for information retrieval and analysis. Data exists in many forms, ranging from structured tables and databases to unstructured text, images, and videos. While humans can naturally interpret and connect this information using context, prior knowledge, and intuition, machines lack this inherent understanding. This has motivated the development of semantic technologies and knowledge graphs, which represent entities, their relationships, and contextual information in a machine-readable format. These systems enable automated reasoning, disambiguation, and linking across heterogeneous sources.

Despite these advances, entity linking, the task of mapping textual or tabular mentions to canonical entities in a knowledge graph, remains challenging. Ambiguities, variations in naming, and incomplete context often lead to errors in automated linking. Existing Semantic Table Interpretation (STI) approaches provide useful reference points, but there remains a need to evaluate newer transformer-based tools on practical tabular datasets, where structure and semantics interact in complex ways.

To tackle these challenges, this thesis focuses on the ReFinED entity linking system, an open-source tool developed by Amazon Science. ReFinED is designed for efficient entity linking in natural language text. This thesis applies ReFinED systematically to the tabular setting, evaluating its performance across a range of SemTab benchmarks and domain-specific datasets, adapting it through supervised fine-tuning, and integrating it as a reconciliation provider within an interactive annotation ecosystem.

1.2 Goals of the Thesis

The goal of this thesis is to explore and evaluate ReFinED in a structured, practical, and systematic manner. Its performance is assessed on tabular datasets from the SemTab challenge: `Round1_T2D`, `Round3_2019`, `Round4_2020`, `2T_Round4`, `HardTablesR2`, `HardTablesR3`, `HTR1`, and `HTR2`, as well as three domain-specific datasets: `Companies`, `Movies`, and `Spend Network` (SN). Together, these eleven datasets provide a standardised basis for comparison with existing STI approaches across a range of table types, difficulty levels, and domain characteristics.

This work has three objectives, each corresponding directly to the research questions in section 1.3. First, it establishes a baseline evaluation of ReFinED by benchmarking its performance across all eleven datasets, examining cell-level versus row-context inference strategies, and comparing results against existing STI approaches. Second, it explores domain adaptation through supervised fine-tuning on domain-specific tabular data, evaluating whether targeted adaptation improves performance over the general-purpose pre-trained model on both in-domain and out-of-domain datasets. Third, it investigates the computational trade-offs of large-scale tabular entity linking, analysing how inference strategy, batch size, and prediction mode affect throughput and memory consumption in practice.

Beyond empirical evaluation, the thesis demonstrates the practical applicability of ReFinED through its integration with Koala-UI¹, an open-source workspace for tabular data management and entity reconciliation, where ReFinED was connected as an additional reconciliation provider, demonstrating its practical applicability in an interactive tabular annotation workflow.

¹Koala-UI repo: https://github.com/enRichMyData/koala_ui

1.3 Research Questions

This thesis is structured around three primary research questions, each targeting a distinct aspect of ReFinED’s applicability to tabular entity linking:

RQ1 — Benchmarking: How does ReFinED, a transformer-based neural entity linking model, perform on tabular CEA benchmarks compared to existing STI approaches, and to what extent does row-level contextual information improve disambiguation over cell-only inference?

RQ2 — Adaptation: How does supervised fine-tuning on domain-specific tabular data affect ReFinED’s entity linking performance compared to the general-purpose pre-trained model, on both in-domain and out-of-domain datasets?

RQ3 — Efficiency: What are the computational trade-offs of deploying ReFinED for large-scale tabular entity linking, and how do inference strategy, batch size, and prediction mode affect throughput and memory in practice?

1.4 Statement of Contributions

While the core ReFinED architecture and pre-trained weights were developed by Amazon Science, this thesis is built upon a significant expansion and adaptation of the original codebase. The research is conducted via a development fork of the repository², with the following components constituting the original work and primary contributions of the author:

- **System Infrastructure and Orchestration:** Designed and implemented a containerisation strategy for the model to ensure a reproducible environment. This included the creation of `docker-compose.yml`, specialised `docker-compose.gpu.yml` for NVIDIA-container-toolkit integration, and environment configuration templates (`.env.template`).
- **API Development (`api/` folder):** Developed a complete FastAPI-based REST service from scratch. This original codebase handles the transition of ReFinED from a local script to a deployable web service, featuring asynchronous task management for large-scale table reconciliation.
- **Evaluation and Research Framework (`my_tests/` folder):** Built a custom suite of testing and benchmarking scripts. This includes the automated evaluation pipeline for the 11 SemTab datasets, scripts for generating precision-recall dynamics, and the log management system that produced the experimental data in chapter 5.
- **Domain Adaptation:** Curated the domain-specific training sets for *Movies* and *Companies* and executed the fine-tuning lifecycle contributing to the comparative ablation studies discussed in chapter 6.
- **Integration:** Implemented the bridge between the ReFinED API and the Koala-UI platform, enabling interactive entity linking within a functional user interface.

² Original ReFinED repo: <https://github.com/amazon-science/ReFinED>
Forked ReFinED repo: <https://github.com/enRichMyData/ReFinED>

1.5 Thesis Structure

Chapter 2 provides the theoretical background, reviewing semantic technologies, knowledge graphs, and the evolution of entity linking. It presents existing approaches to tabular entity linking and introduces the SemTab challenge and evaluation metrics used throughout this thesis.

Chapter 3 introduces the benchmark datasets selected from the SemTab challenge series, describing their structural characteristics, preprocessing steps, and the evaluation scope defined for this research.

Chapter 4 details the computational environment, the model configuration, and the inference strategy used in all experiments. The automated evaluation pipeline, the back-end REST API service built to expose ReFinED as a web service, and its integration with the Koala-UI reconciliation platform are all shown and explained.

Chapter 5 presents the experimental design and benchmark results, examining ReFinED’s performance across all datasets, the impact of row-level contextual inference on disambiguation, and a comparative analysis against existing STI approaches.

Chapter 6 explores domain-specific adaptation through supervised fine-tuning, investigating how training on domain-specific tabular data affects performance on both in-domain and out-of-domain datasets.

Chapter 7 contextualises the empirical results within the broader landscape of tabular entity linking, reflecting on the strengths and limitations of ReFinED, dataset-specific observations, and the practical implications of deploying large-scale transformer models as web services.

Chapter 8 summarises the thesis contributions and outlines the direction of future research.

Chapter 2

Theoretical Background and Related Work

This chapter provides the theoretical and conceptual foundations for this thesis. It introduces key concepts in semantic technologies and knowledge graphs, the challenges of entity linking, and specific considerations for linking entities in tabular data. Additionally, an overview of existing approaches to tabular entity linking, including ReFinED and selected STI systems used for comparison is provided. This chapter then concludes with the introduction of the SemTab challenge, which forms the main basis of the datasets used in later chapters.

2.1 Semantic Technologies and Knowledge Graphs

Semantic technologies and knowledge graphs are essential to understanding the mechanics of entity linking systems, as they provide the formal structures upon which linking operates. Knowledge graphs define entities, their relationships, and the contextual metadata required to resolve ambiguities in linking tasks.

While traditional relational databases are optimised for structured data with predefined schemas, the modern data landscape is dominated by information lacking a consistent format. Gandomi and Haider [1] has long highlighted that unstructured data, such as news articles and social media, constitutes up to 95% of “Big data” volume. Because this information is frequently noisy and unreliable, it presents significant challenges to traditional automated analysis. Consequently, semantic alignment of unstructured content with structured knowledge bases has become a primary objective within the field of machine learning and information retrieval.

2.1.1 Ontologies and Knowledge Graphs

To address this, semantic technologies provide a framework for structuring and querying knowledge. The goal is to allow machines to move beyond simply reading data to “understanding” it through explicit descriptions of what the data represent and how it relates to other information.

At the core of this field are **ontologies** and **knowledge graphs**. An ontology defines the concepts, classes, and constraints of a domain in a formal manner, providing a shared conceptualisation interpreted by both humans and machines [2]. Knowledge graphs, in turn, are structured representations of real-world entities modelled as graphs where nodes represent entities and edges represent semantic relationships [3].

2.1.2 The Semantic Technology Stack

The implementation of these concepts relies on a layered stack of standards developed by the W3C, ranging from basic data models to complex logical axioms.

The RDF Triple. One of the core building blocks of this framework is the **Resource Description Framework (RDF)**¹. Introduced by W3C as a general model for machine-processable information, RDF expresses knowledge in the form of **triples**, consisting of *a subject, a predicate, and an object*. For example, the statement “Berlin is the capital of Germany” is represented using URIs as unique identifiers:

`<uri/Berlin> <uri/capitalOf> <uri/Germany>`

This structure forms a graph of interconnected facts, a web of data, based on the principles of Linked Data². To support richer semantic capabilities, **RDF Schema (RDFS)**³ introduces class and property hierarchies, allowing entities like `<uri/Berlin>` to be defined as an instance of the `<uri/City>` class, while `<uri/Germany>` is defined as an instance of the `<uri/Country>` class. Such classification allows the system to use type information when resolving ambiguities.

Logic and Querying. For further expressiveness, the **Web Ontology Language (OWL 2)**⁴ supports complex logical constructs such as class disjointness and property symmetry. These axioms allow systems to formally express that a relationship may only occur a specified number of times or that instances of one class must belong to another. Interacting with these structures is made possible via **SPARQL**⁵, a SQL-like language

¹<https://www.w3.org/TR/owl2-primer>

²<https://www.w3.org/DesignIssues/LinkedData.html>

³<https://www.w3.org/TR/rdf-schema>

⁴<https://www.w3.org/TR/owl2-primer>

⁵<https://www.w3.org/TR/sparql11-query>

used to extract, aggregate, and reason over information in knowledge graphs.

Implementation Tools. Working with these standards requires specialised frameworks. In Python, the primary language of this thesis, **RDFlib**⁶ is an essential tool for manipulating RDF graphs, and executing SPARQL queries. While not used for large-scale model inference in this work, the use of RDFlib and Java-based frameworks like **Apache Jena**⁷ proved to be essential tools for gaining hands-on experience with semantic technologies. The use of these tools provided a strong conceptual understanding of RDF structures, namespaces, ontologies, SPARQL querying, and knowledge that directly informed the design and evaluation of the ReFinED entity linking experiments.

2.2 Entity Linking: Concepts and Challenges

Entity linking is a central component of semantic technologies and knowledge-based systems and forms the core focus of this thesis. In order to interpret the experimental results and design decisions presented in later chapters, an understanding of its fundamental components and challenges is necessary.

At its core, entity linking is the task of identifying mentions of real-world concepts in data and associating them with their corresponding canonical entities in a knowledge base or knowledge graph. Mentions may appear in unstructured text, semi-structured content, or structured data such as tables. For example, the mention “Paris” could refer to the capital of France, a person named Paris, or another entity entirely, depending on the context. The goal of entity linking is to resolve such mentions to the correct, unique identifier representing the intended entity.

Formally, entity linking can be viewed as a mapping function

$$f : M \rightarrow E \cup \{NIL\},$$

where M represents the set of detected mentions in the input data, E represents the set of entities in a target knowledge base, and NIL denotes mentions that do not correspond to any known entity. The inclusion of NIL reflects the **open-world assumption** (OWA): the knowledge base is assumed to be incomplete, and entities may simply be missing from it. This contrasts with the **closed-world assumption** (CWA), where statements that cannot be derived from the knowledge base are treated as false [4]. In entity linking systems, this is often reflected in constrained settings where a link is selected from a predefined set of candidates rather than allowing explicit NIL predictions. While the idea is simple, this mapping presents several challenges in practice, especially when automated.

⁶<https://rdflib.readthedocs.io>

⁷<https://jena.apache.org>

Figure 2.1 illustrates a simple process of entity linking applied to tabular data.

It is important to distinguish between *NIL* entities and non-entity cells. A *NIL* prediction indicates that no suitable entity in the knowledge base can be assigned to a detected mention, either because the entity is missing from the knowledge base or because the available context is insufficient for reliable disambiguation. In contrast, cells that contain literal data (e.g., numbers, dates, or boilerplate text) are not considered mentions (*M*) and fall outside the scope of *NIL* classification.

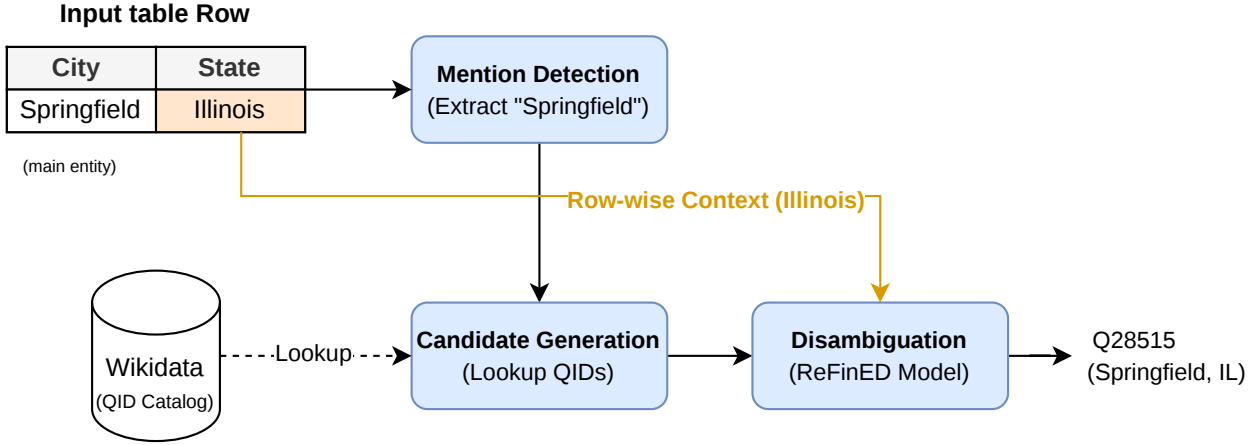


Figure 2.1: Process of Entity Linking: The model utilises row-wise context (neighbouring cells) to resolve surface form ambiguity.

2.2.1 Surface Forms and Ambiguity

A key distinction in entity linking is the difference between **surface forms** and **canonical entities**. Surface forms are textual representations of entities as they appear in the data, whereas canonical entities correspond to unique identifiers in a knowledge base, typically represented using URIs. For example, “USA”, “United States”, and “America” are all surface forms referring to the same real-world concept, whose canonical entity is the unique Wikidata identifier Q30. This many-to-one relationship between surface forms and canonical entities complicates entity linking, as string-matching approaches are insufficient. Effective systems must therefore rely on semantic representations that capture meaning beyond simple string similarity.

The Challenge of Ambiguity. One of the central challenges in this mapping is ambiguity. Human language contains numerous words and phrases that can represent multiple meanings, described through homonymy and polysemy. Homonymy refers to cases where a term has multiple unrelated meanings (e.g., a “bat” as an animal or sports equipment), while polysemy refers to terms with related but distinct meanings (e.g., “line” as a row, a

connection, or a geometric object). As noted by Limaye et al. [5], text similarity is often a “noisy signal” because a single mention can refer to many distinct entities. Without explicit mechanisms to interpret context, an entity linking system may incorrectly associate a mention with an unintended entity, leading to further errors in downstream applications.

2.2.2 Transformers and Embeddings

Modern entity linking approaches increasingly rely on machine learning and deep learning techniques to capture contextual information. The transition from symbolic matching to neural entity linking is based on **embeddings**, dense numerical vectors that represent semantic meaning. This concept was pioneered by Bengio et al. [6], who proposed learning distributed representations of words in a continuous vector space. Unlike sparse representations such as one-hot encoding, embeddings place semantically similar concepts together in proximal regions in this space. This allows influential models such as Word2Vec [7] to recognise that “USA” and “United States” are related without the need for explicit rules.

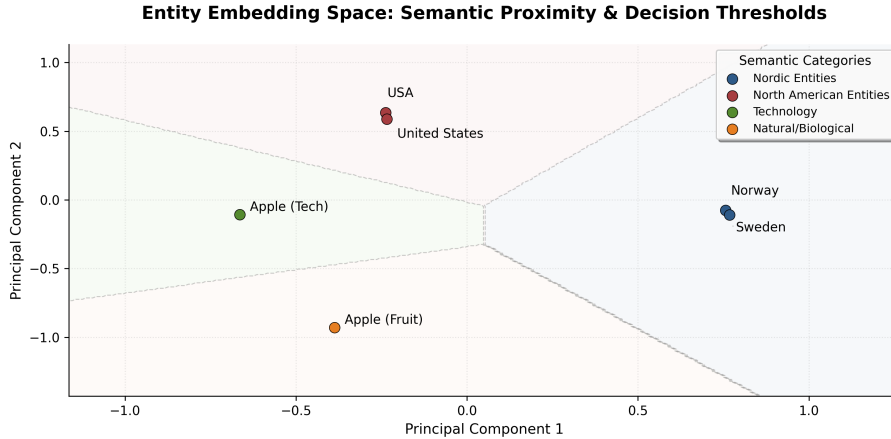


Figure 2.2: 2D PCA projection of entity embeddings with categorical decision boundaries.

As illustrated in fig. 2.2, these high-dimensional embeddings can be projected into a two-dimensional space⁸ to reveal the underlying semantic topology. The dashed lines represent decision boundaries (derived from a Voronoi partition), which define the boundaries of the semantic regions where one entity category shifts to another. This spatial separation is what enables the system to distinguish between identical surface forms, such as the technological and biological interpretations of “Apple”, based on their surrounding context.

⁸Using Principal Component Analysis (PCA), a dimensionality reduction technique that projects high-dimensional data onto its axes of greatest variance.

While a one-hot encoding for “Norway” might be a long, sparse vector of zeros with a single “1” at a specific index $[0, 0, 1, 0, \dots]$, a dense embedding represents the same concept as a compact array of floating-point numbers, such as $[0.12, -0.54, 0.89]$. These values do not have a fixed human definition but are learnt during training to capture the characteristics of entities.

The Transformer Architecture. The dominant framework for generating these embeddings is the Transformer [8], a neural architecture that builds context-aware representations by calculating the relationships between all tokens in a sequence at once. Rather than processing tokens in isolation, it allows the model to interpret a token based on the global context of the entire input, ensuring that the resulting embedding captures the specific sense of how a word is being used. Its core innovation is the **self-attention mechanism**, which allows the model to dynamically weight the relevance of every other token in a sequence when computing the representation of a given token. This means that the embedding for a word like “Paris” will differ depending on whether it appears in context of travel, fashion, or a person’s name, a critical property for entity disambiguation. Unlike earlier recurrent architectures that processed tokens sequentially, the Transformer processes all tokens in parallel, making it significantly more scalable.

Bidirectional Encoding and RoBERTa. A key development built on the Transformer is a bidirectional encoding, where a model considers both the left and right contexts of a token simultaneously rather than processing text in a single direction. **RoBERTa** [9] is a widely adopted implementation of this approach, built on the bidirectional BERT architecture [10] with improved pre-training. In particular, RoBERTa removes the next prediction objective, uses dynamic masking during training, and is trained on significantly larger datasets. These changes lead to more robust and stable contextual representations in practice. By reading the full surrounding context at once, RoBERTa produces richer, more context-sensitive embeddings that are well-suited to disambiguation tasks where the same surface form can refer to many different entities.

Systems such as ReFinED [11] build directly on RoBERTa to perform end-to-end linking, as described in detail in section 2.4.2

2.3 Entity Linking for Tabular Data

While the fundamental goals of entity linking remain the same across different data formats, tabular data presents a unique set of challenges and opportunities compared to unstructured text. In a standard sentence, linguistic features such as grammar, syntax, and parts of speech provide rich contextual clues to what is said. In contrast, tables are often “language-independent” and lack such narrative structures, instead relying on a two-dimensional layout to convey meaning.

2.3.1 Structural Context and Data Quality

A big challenge when it comes to tabular data is the absence of linguistic context. A cell with the word “Paris” would not usually be accompanied by a full sentence clarifying if it refers to a city or a person. Instead, the system must infer context from horizontal relationships (other cells in the same row) and vertical relationships (other cells in the same column). For example, if a row contains “Paris” alongside “France” and “2.1 million”, the horizontal context strongly suggests that “Paris” refers to the French capital city. Similarly, if the column header is “Birthplace”, it provides a semantic type constraint that narrows down the candidate entities to locations.

Column headers are particularly useful as they often act as a label for the class of entities in that column (e.g., “City”, “Species”, “Title”). In semantic terms, the relationship between a subject column and a property column can often be mapped to a prediction in a knowledge graph. Disambiguating a cell therefore often involves understanding both the column’s intended type and the relationships expressed across rows.

Tables found online also come with their own quality issues and structural challenges. They may contain missing values, inconsistent formatting, or abbreviations that do not match the canonical labels in a knowledge base. Without clear domain metadata, a system often has no choice but to piece together meaning from surrounding cells in a column.

2.4 Approaches to Tabular Entity Linking

The methodologies used to solve the challenges described in the previous sections have evolved significantly over the last decade. Early approaches relied heavily on lookup-based heuristics and string matching, while modern systems utilise deep learning to capture structural semantics. This section surveys representative systems across this spectrum, from supervised re-ranking pipelines and encoder-based models to LLM-based approaches, before introducing ReFinED, the system adopted in this thesis.

Supervised Re-ranking: Alligator. Alligator [12] is a modular supervised re-ranking framework for tabular entity linking that utilises a human-in-the-loop philosophy for semi-automated table annotation. Unlike end-to-end neural models, Alligator functions as a supervised re-ranker, using a multi-layer Feed-Forward Neural Network (FFNN) to calculate a normalised matching score ρ for each candidate entity. This score is derived from a variety of features, including textual similarity (via LamAPI [13]⁹), column type predictions, and semantic consistency within the table rows. The matching score ρ is then combined with a confidence parameter δ , defined as the difference between the scores of the top two candidates, into a global confidence score $\omega \in [0, 1]$. This confidence score

⁹<https://github.com/unimib-datAI/lamAPI>

determines whether to establish a link or flag it for human review, allowing the system to prioritise user attention on low-confidence links. In practice, users need to review no more than 30% of the data to reach F1 scores above 0.98, while the remaining automated links maintain near-perfect precision [12]. However, Alligator’s performance is dependent on the quality of the initial candidate generation and the availability of pre-defined schema feature, as well as the underlying quality of LamAPI, which handles candidate lookup via Elasticsearch-based indexing.

Heuristic-based: DAGOBABH. DAGOBABH [14] is an end-to-end semantic table interpretation system developed jointly by Orange and EURECOM, evaluated across multiple editions of the SemTab challenge. Its annotation system follows four sequential stages: table pre-processing, entity lookup, candidate-pre-scoring, and final cell-entity annotation informed by column-type and column-property predictions. Candidate retrieval is handled by an Elasticsearch-based lookup service supporting both Wikidata and DBpedia, with aliases enriched from additional KG properties to improve coverage. Disambiguation uses a scoring function that combines lexical similarity between cell mentions and entity labels with contextual information from neighbouring cells and related nodes in the knowledge graph. A notable improvement in DAGOBABH SL 2021 was the extension of graph context from one-hop to two-hop predicate paths, which significantly improved disambiguation for entities whose relevant context nodes were not directly adjacent in the KG. DAGOBABH represents a mature heuristic-based pipeline approach, offering interpretable scoring and strong coverage, but its reliance on Elasticsearch indexing and hand-crafted scoring functions limits its adaptability to unseen entity types or mentions out of its vocabulary.

Heuristic-based: LinkingPark. LinkingPark [15] is an integrated semantic table interpretation system developed by Microsoft Research, submitted to SemTab 2020 where it ranked among the top-3 systems across all rounds. Its entity linking pipeline consists of three stages: candidate generation via the Wikidata MediaWiki API combined with a spelling corrector and fine-grained Elasticsearch index, followed by an iterative coarse-to-fine entity disambiguation algorithm that exploits both column-level type consistency and row-level property relatedness to resolve ambiguous mentions. Unlike purely neural approaches, LinkingPark combines lexical similarity, entity priors, and structural table context through a hand-crafted scoring function, achieving CEA F1 scores of 0.985 on Round4 and 0.810 on the adversarial 2T dataset, the latter being particularly relevant as a reference point for ReFinED’s performance on the same dataset.

Encoder-based: TURL. TURL (Table Understanding Through Representation Learning) [16] introduces the pre-training and fine-tuning paradigm to relational web tables, applying a structure-aware transformer encoder that models the row-column structure of tables explicitly. During pre-training on large-scale Wikipedia tables, TURL learns deep contextualised representations through a Masked Entity Recovery (MER) objective, which

trains the model to recover masked entity cells from their surrounding table context. This design allows a single pre-trained model to be fine-tuned for a range of downstream tasks including entity linking, column type annotation, and relation extraction with minimal task-specific adaptation. For entity linking, TURL generates candidate entities via the Wikidata Lookup service and re-ranks them using contextualised cell representations that fuse entity mention embeddings with table metadata. As noted by Belotti et al. [17], however, TURL’s entity linking formulation is constrained to the set of provided candidates and offers no mechanism for abstaining or returning a *NIL* prediction, meaning it will always assign an entity even when no correct candidate is present. This closed-world assumption limits its ability in open-world annotation scenarios where NIL-awareness is required.

Iterative Refinement Pipelines. Beyond individual top-ranking finishes, **MTab** [18] and **JenTab** [19] have demonstrated competitive performance across multiple SemTab editions. MTab uses a probabilistic approach, integrating fuzzy entity search with a join-based strategy to estimate likelihoods from entity-type co-occurrences in the KG. On the other hand, JenTab uses a multi-layered ensemble approach that iteratively reduces candidate entities by leveraging graph-based reasoning to validate schema consistency. While highly effective within SemTab, these systems represent traditional, modular pipelines that contrast with the more streamlined, end-to-end neural architecture described in this section.

Early Context-Based Models. A key shift toward modern STI logic is represented by models such as **TableMiner+** [20], which introduced an incremental bootstrapping approach using high-confidence “seed” annotations to guide the interpretation of remaining table content. This established the foundational principles of utilising structural context to resolve ambiguity, as TableMiner+ demonstrated that the usage of textual context both around and outside tables, such as captions and paragraphs, could be used to improve accuracy.

LLM-based Approaches. With the recent rise of Large Language Models (LLMs), the landscape of entity linking has shifted significantly. Belotti et al. provide a direct comparative evaluation of both heuristic-based and LLM-based STI approaches on CEA benchmarks, covering decoder-based models such as **TableLlama** [21] and generative models such as **GPT-4o** and **GPT-4o-mini**. While these approaches demonstrate strong performance, particularly on semantically rich web tables, several key distinctions remain relevant when comparing them to specialised linkers like ReFinED.

Among the systems evaluated by Belotti et al., TableLlama takes a decoder-only approach, built on Llama 2 (7B) with LongLoRA to handle long-context inputs, and instruction-tuned across a wide range of table-related tasks [21], including entity linking evaluated as

CEA in the STI context. GPT-4o and GPT-4o-mini represent the large-scale generative paradigm, relying entirely on knowledge encoded in their weights to resolve entity mentions without access to an explicit candidate set or external knowledge base. This makes them flexible and broadly capable, but at the cost of reproducibility and interpretability, although there is no guarantee that a given prediction is grounded in a specific, versioned knowledge graph entry.

Several limitations distinguish LLM-based approaches from specialised linkers. First, LLMs risk producing hallucinated or outdated entity mappings, as their knowledge is static and bounded by their training cut-off. Second, structure-awareness is limited: while LLMs can process serialised tabular data, large or structurally complex tables exceed context window limits. TableLlama is constrained to 8,192 input tokens [21, p. 6028], requiring truncation for larger inputs, while even GPT-4o’s larger context window does not eliminate the challenge of reasoning over wide, multi-column tables. Third, and most critically for benchmark evaluation, neither TableLlama nor GPT-4o-based approaches evaluated by Belotti et al. support *NIL* prediction [17]. They will always assign an entity even for ambiguous mentions outside the vocabulary. This is compounded by the fact that TableLlama operates over a provided candidate shortlist of at most 50 candidates [17], meaning that when the correct entity is absent from the list, an incorrect entity is returned regardless. ReFinED, by contrast, produces a normalised confidence score and can abstain via *NIL*, which is particularly valuable in human-in-the-loop workflows where over-linking is costly. Finally, autoregressive LLMs incur substantially higher per-sample cost than specialised linkers, making them less practical for large-scale tabular entity linking pipelines.

Neural Approach: ReFinED. More recent methodologies have shifted toward end-to-end neural architectures. **ReFinED** [11] is a clear example of this shift, offering an efficient and scalable approach to the problem. Its main advantage lies in the ability to perform *mention detection*, *entity typing*, and *entity disambiguation* simultaneously within a single forward pass. By eliminating the need for separate, computationally expensive candidate generation and feature engineering steps, ReFinED achieves inference speeds over 60 times faster than many existing competitive models [11].

ReFinED uses a bi-encoder architecture built on RoBERTa to map mentions and knowledge base entries into a shared mathematical vector space. The model is trained so that a mention and its correct entity are placed close together in this space, while unrelated entities are pushed far apart, similar to the embedding space represented in fig. 2.2. This similarity is calculated by projecting the mention embedding $\mathbf{m}_i$ and the entity description embedding $\mathbf{d}_j$ through linear layers (f_2 and f_3) and computing their dot product:

$$\psi(e_j, m_i) = f_2(\mathbf{m}_i) \cdot f_3(\mathbf{d}_j)$$

Here, e_j represents a candidate entity from the knowledge base, and the functions f_2 and f_3 act as the final mapping to the shared space. This **description score** is combined with two additional signals: a **fine-grained entity typing score**, which ensures the entity matches the predicted category (e.g., “City” vs “Person”), and a **global entity prior**, which captures the general popularity of an entity given a surface form (i.e., how likely a mention such as “Paris” is to refer to a given entity based on how frequently that entity is linked to that surface form in the training data). These three signals are combined into a final score ω which is used to select the most likely entity or *NIL* if no candidate is sufficiently confident [11]. Specifically, a special *NIL* candidate with a fixed score of zero is included alongside all real entity candidates. A real entity is predicted only when its ω exceeds this threshold, meaning *NIL* is predicted only when all candidate ω scores are negative. This combination enables zero-shot linking, the model can link to entities it never saw during training by calculating semantic similarity between the mention context and the entity’s description in Wikidata. This thesis adopts ReFinED as its primary system, evaluating how this neural approach scales and performs when applied to the diverse tabular datasets of the SemTab benchmark suite, as described in chapter 4.

2.5 The SemTab Challenge

The Semantic Web Challenge on Tabular Data to Knowledge Graph Matching (SemTab)¹⁰ is an annual competition that has become a big benchmarking platform for evaluating table-to-knowledge graph matching systems [22]. By providing standardised datasets and evaluation frameworks, SemTab enables the systematic comparison of diverse approaches, from heuristic-based systems and language model approaches to neural architectures such as ReFinED

2.5.1 Challenge Structure

The SemTab challenge is structured around three core annotation tasks:

- **Cell Entity Annotation (CEA):** Linking a specific cell to its corresponding entity in a knowledge graph. This is the primary task focused on in this thesis.
- **Column type Annotation (CTA):** Assigning a semantic type to a column of entities.
- **Column Property Annotation (CPA):** Identifying a semantic relationship between two columns.

¹⁰<https://www.cs.ox.ac.uk/isg/challenges/sem-tab/>

The participating systems are evaluated using standard information retrieval metrics, formally defined in section 2.5.2, applied at the cell level for the CEA task. By using these task-specific definitions [22], the challenge ensures that a system’s ability to handle *NIL* entries, which are often excluded from the ground truth but included in precision calculations, is accurately reflected in the final leaderboard. It is worth noting that systems participating in SemTab are typically purpose-built and optimised specifically for its datasets, often relying on challenge-specific training, as alongside testing datasets, SemTab also provides training datasets. This stands in contrast to the general-purpose zero-shot approaches such as ReFinED, which enter the same benchmarks without any dataset-specific adaptation, making direct score comparison difficult to interpret without accounting for this distinction.

2.5.2 Evaluation Metrics and Trade-offs

Entity linking systems must balance competing evaluation objectives. **Precision** (P) measures predictions quality, while **Recall** (R) measures ground truth coverage [23, p. 155]. These are formally defined as:

$$P = \frac{tp}{tp + fp}, \quad R = \frac{tp}{tp + fn} \quad (2.1)$$

Here, tp , fp , and fn represent true positives, false positives, and false negatives, respectively. These quantities inherently trade off against one another [23, p. 156]. In practice, improving precision often comes at the cost of recall. For instance, a system may achieve high precision by applying a strict confidence threshold, only linking mentions it is certain about, but this typically results in lower recall by ignoring ambiguous mentions.

To evaluate the balance between these objectives, the **F1-score** (“the balanced F-measure”) is used as the harmonic mean of precision and recall [23, p. 156]:

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} \quad (2.2)$$

These trade-offs are especially relevant in tabular settings, where the SemTab challenges use specific variants of these metrics to evaluate Cell Entity Annotation (CEA) [22]. In these scenarios, incomplete context and noisy data are common. Furthermore, as catalogues and knowledge bases are “rarely complete or perfect” [5], a system’s ability to refrain from linking (recognising a *NIL* entity when a link is missing from the underlying catalogue) is just as critical to its overall score as its accuracy on known entities.

While the **Accuracy** metric is often used interchangeably with Recall in CEA tasks (measuring the fraction of correctly labelled target cells), the F1-score remains the primary ranking metric in the SemTab challenge [22]. This is because accuracy alone can be less descriptive in skewed data distributions where *NIL* entities are prevalent [23, p. 155], as seen in the adversarial and real-world datasets evaluated in this study. In such cases, a system might achieve a misleadingly high accuracy by simply defaulting to *NIL* for unknown entries, whereas Precision and F1-score provide deeper insight into the model’s actual retrieval performance on known entities.

Understanding these challenges provides the foundation for evaluating the performance of ReFinED across the benchmark datasets described in the following chapters.

2.5.3 Relevance to this Thesis

The SemTab challenge provides more than just data, it offers a variety of tracks (such as the Accuracy Track or the Datasets Track) and diverse datasets that simulate real-world web noise. Using SemTab benchmarks allows this research to validate whether ReFinED’s neural approach truly offers a viable alternative to heuristic baselines in the face of missing metadata and high ambiguity.

Chapter 2 conclusion

This chapter has outlined the development of semantic technologies, the linguistic challenges of entity linking, and the structural difficulties unique to tabular data. We have examined two different methodologies: heuristic-based STI approaches and the end-to-end neural architecture of ReFinED. Finally, we introduced SemTab as the standardised evaluation framework for this research. The following chapter describes the specific datasets used in this evaluation, detailing their selection, structural characteristics, and the preprocessing steps required to ensure consistent input to the systems.

Chapter 3

Datasets

Building on the theoretical framework of the SemTab challenge and its associated metrics presented in chapter 2, this chapter describes the specific datasets used in this thesis. The focus shifts from the competition’s general objectives to the structural and semantic properties of the tables themselves. By detailing the selection criteria and the preprocessing steps required, this chapter provides the grounding necessary to interpret the experimental results and performance trade-offs discussed later in this thesis.

3.1 SemTab Selection and Data Characteristics

To provide a comprehensive evaluation of ReFinED’s performance, this research uses data from several different iterations of the SemTab challenge. Rather than processing every available round, specific datasets were selected to create a representative spectrum of difficulty, ranging from clean web tables to highly adversarial data with noise. This selection allows us to investigate how the model’s transformer-based architecture adapts to varying levels of ambiguity, table size, and metadata availability.

3.1.1 Standard Web Tables

These datasets represent the foundational tasks of the challenge, consisting primarily of relational tables extracted from the web.

The **Round1 (T2D)** dataset, based on the T2D Gold Standard [24], serves as the initial baseline. These tables are generally well-structured and link mentions to the DBpedia knowledge graph. While they are smaller than later datasets, they are essential for validating the basic configuration of the linking tools.

Following this, the **Round 3 (2019)** and **Round 4 (2020)** datasets are used to test scalab-

ility. As noted in the respective challenge results [22, 25], these rounds introduced hundreds of thousands of target cells, consisting of 406,827 targets in 2019 linked to DBpedia and over 1.6 million targets in 2020 linked to Wikidata. This transition in 2020 provides a robust measure of how neural architectures like ReFinED handle both large-scale lexical heterogeneity and the complexities of different knowledge bases.

To assess computational efficiency and robustness, this research uses the 2T (Tough Tables) dataset [26]. Designed to bridge the gap between small web snippets and real-world legacy tables, 2T features significantly higher row counts and denormalised structures that challenge heuristics relying on column-wide context. It evaluates three adversarial mechanisms: **TOUGH_HOMO** (high-ambiguity homonyms), **TOUGH_MISSP** (misspelled mentions and noise), and **TOUGH_SORTED** (structural manipulation). Using the **2T_Round4** version, which consists of 180 tables and 667,244 target cells, we measure the inference speed and VRAM consumption of ReFinED’s transformer engine under conditions mirroring industrial-scale relational data.

3.1.2 HTR1 and HTR2

In addition to the massive 2T dataset, this research evaluates ReFinED using the Hard Table benchmarks, **HTR1** and **HTR2**. These datasets are sourced from the SemTab challenge¹, and were accessed via a standardised benchmark suite curated by Avogadro and Rauniyar [27].

The **HTR** datasets are designed to test the limits of neural entity linkers by using the adversarial design philosophy established by Cutrona et al. [26]. Empirical inspection of both **HTR1** and **HTR2** tables confirms the presence of these mechanisms, specifically data including character-level noise and encoding artefacts (e.g., *MÄ¼rren* for *Mürren*), synthetic typos (e.g., *Haarlgm* for *Haarlem*), and high-ambiguity homonyms like common geographic names. This provides a rigorously verified environment for evaluating disambiguation performance via fixed candidate lists (**mention_to_qid**)

In this study:

- **HTR1** consists of 200 tables with 1,406 target cells.
- **HTR2** consists of 426 tables² with 1,983 target cells.

A significant feature of this specific benchmark version is the inclusion of pre-computed candidate mappings (**mention_to_qid**). This allows for a focused evaluation of ReFinED’s ability to select the correct Wikidata QID from a fixed list of candidates, directly measuring

¹<https://www.cs.ox.ac.uk/isg/challenges/sem-tab/>

²HTR2 source file contains 457 CSV files, only 426 are in the evaluation

its disambiguation performance under adversarial conditions.

3.1.3 HardTables and Adversarial Mechanisms

This research further uses the full-scale HardTables datasets from the SemTab 2021 competition (Rounds 2 and 3) [28]. These datasets represent the **Automatically Generated (AG)** track: a synthetic dataset with tables generated automatically by SPARQL queries, representing a significant scaling of the adversarial design philosophy established by the Tough Tables benchmark [26].

While they employ the same core mechanisms described in the previous section, **TOUGH_HOMO**, **TOUGH_MISSP**, and **TOUGH_SORTED**, they do so at a much larger volume to stress-test the robustness of entity linking algorithms in general. To ensure experimental reproducibility, these specific iterations were verified against the official archival records³.

- **HardTablesR2** provides 47,440 target cells, focusing on high-volume ambiguity.
- **HardTablesR3** provides 58,949 target cells, introducing more complex structural variations.

3.1.4 Domain-Specific Datasets

In addition to the evaluation benchmarks, two domain-specific subsets from Wikidata were used for initial model testing and later used in the fine-tuning experiments described in chapter 6 to test whether ReFinED’s zero-shot capabilities could be enhanced by domain-specific training. These datasets include:

- **Movie Dataset:** A collection of tables containing film titles, release dates, genres, countries, languages, and directors.
- **Company Dataset:** Tables focused on corporate entities, including company names, founding years, website links, employee counts, founders, industry types, countries, and headquarters.

This thesis also uses the **Spend Network (SN)** dataset from the enRichMydata (EMD) project [29]. Unlike the SemTab datasets, the SN data originates from real-world procurement records aggregated from over 180 million entries across EU and international sources [27], characterised by fragmented, inconsistent strings, multilingual variations, and non-standard abbreviations, although the subset evaluated in this thesis consists of only 1,000 sampled rows. A notable feature of this dataset is the high prevalence of *NIL* entities. Approximately 22.9% of the ground truth consists of *NIL* cases [29], reflecting realistic

³HardTables (R2 and R3): <https://zenodo.org/records/6154708>

industrial ambiguity where many entities do not exist in canonical reference graphs like Wikidata. It is used here as a challenging real-world counterpart to the domain-specific datasets above (where Movies and Companies represent relatively controlled conditions) the SN dataset stress-tests the system against the ambiguity and noise characteristic of production data.

3.2 Dataset Selection and Scope

The datasets used in this thesis span a range of difficulty levels, supporting a comparative evaluation of ReFinED’s automated neural approach against existing STI models. While the SemTab challenge provides an extensive collection of benchmarking datasets, this research focuses on a subset that targets scalability, disambiguation under ambiguity, and robustness to noise.

The scope of this evaluation is defined by four primary objectives:

1. **High-Volume efficiency:** Using the large-scale `2T_Round4` (667,244 linkable cells) and `Round 4 (2020)` (994,921 linkable cells) datasets to verify if ReFinED maintains inference efficiency when processing nearly one million target cells. These datasets provide the volume necessary to measure how VRAM consumption and time-per-table scale with massive data dimensions.
2. **Disambiguation under Ambiguity:** Using `HTR1`, `HTR2`, `HardTables (R2 and R3)` to evaluate ReFinED’s ability to resolve ambiguity under adversarial conditions, where string similarity is misleading and mentions include character-level noise, synthetic typos, and high-ambiguity homonyms, across approximately 110,000 combined adversarial target cells.
3. **Real-World Robustness:** Applying the `SN` dataset to evaluate how ReFinED handles messy real-world procurement data and high *NIL*-case ratios (22.9%), representing a more challenging real-world counterpart to the controlled domain-specific datasets.
4. **Domain Adaptation:** Using the `Movies` and `Companies` datasets to evaluate whether fine-tuning ReFinED on domain-specific data can improve upon its zero-shot baseline performance, with experiments described in chapter 6.

Exclusions and Constraints: The datasets evaluated in this thesis were obtained incrementally, beginning with simpler baselines and progressively adding more challenging datasets. As a result, several SemTab subsets are not covered, particularly those targeting the DBpedia ontology from earlier rounds, as ReFinED’s zero-shot capabilities rely

on the fine-grained entity descriptions and types found in Wikidata, as well as its specific Wikidata entity set. While **Round1_T2D** presents significant formatting challenges (specifically regarding coordinate alignment and filename-to-target mapping), these were resolved through custom preprocessing, allowing for a full evaluation of the dataset’s 8,418 targets.

3.3 Structural Characteristics

The performance of entity linking systems is fundamentally constrained by the structural “signal” available in input tables, the headers, cell relationships, and metadata that provide the context necessary for disambiguation. This section provides a comprehensive audit of the selected datasets, establishing a baseline for interpreting the experimental results

To ensure experimental consistency, the statistics in Table 3.1 were locally audited against the official ground truth files. While high-level dimensions (total table counts) were adopted from the official SemTab reports [22, 25, 28], specific target cell counts and *NIL* ratios were derived directly from the evaluation scripts. Importantly, this audit distinguishes between targets lacking a ground truth entry and explicit *NIL* mentions. In the evaluated benchmarks, only **2T_Round4** and the **Spend Network** dataset contain true *NIL* cases. For all other datasets, target cells without a valid mapping are excluded from the evaluation to prevent skewed results.

Table 3.1: Structural Characteristics of Evaluated SemTab Datasets.

Dataset	Tables	Targets	Linkable	NIL	NIL Ratio	Challenge	Ref.
<i>Standard</i>							
Round1_T2D	64	8,418	8,079	0	0.0%	Baseline / Web Data	[22]
Round3_2019	2,161	406,827	390,457	0	0.0%	Multi-domain Scale	[22]
Round4_2020	22,207	1,662,165	994,921	0	0.0%	Out-of-KG Detection	[25]
<i>Adversarial</i>							
2T_Round4	180	667,244	663,656	3,588	0.5%	Ambiguity	[26]
HTR1	200	1,406	1,406	0	0.0%	Lexical Noise	[27]
HTR2	426*	1,983	1,983	0	0.0%	Entity Overlap	[27]
<i>Synthesized</i>							
HardTablesR2	1,750	47,440	47,440	0	0.0%	Precision	[28]
HardTablesR3	7,207	58,949	58,949	0	0.0%	Scale	[28]

Note: *Targets* refers to all benchmark cells, *Linkable* refers to targets with a valid Wikidata QID, and *NIL* denotes explicit out-of-KB mentions in the ground truth. * In HTR2, 426 tables were successfully matched from 457 source files. Dataset record IDs are provided in Appendix B.

Table 3.2: Structural Characteristics of Specialised Evaluation Datasets.

Dataset	Tables	Targets	Linkable	NIL	NIL Ratio	Target Domain
Spend Network[29]	1	1,000	771	229	22.9%	Industrial/Finance
Movies	1	1,000	1,000	0	0.0%	Media & Arts
Companies	1	1,000	1,000	0	0.0%	Organisations

Note: These subsets represent *Theme-based* and *industrial data*. The Spend Network dataset is particularly significant as it assesses *NIL* detection capabilities within a real-world business context, where entity overlap with Wikidata is significantly lower.

While the **Round 1 (T2D)** dataset establishes the structural baseline [22], the **Tough Tables (2T)** and **Hard Tables** datasets defined in the 2021 results report [28] provide the adversarial corpus necessary to test system robustness against ambiguity and noise.

The audit reveals a clear distinction in complexity across the evaluation scope. The **2T_Round4** dataset serves as a near-closed-world baseline (0.5% *NIL*) for large-scale disambiguation. This low *NIL* prevalence aligns with the “conservative approach” described by [26], where ambiguous cases were omitted from the ground truth rather than being marked as targets.

In contrast, the high *NIL* ratio observed in the SN dataset (22.9%) suggests an intentional open-world design. While **Round4_2020** and **2T_Round4** focus on large-scale Wikidata linking, they represent different structural challenges. **2T_Round4** provides a low-prevalence explicit *NIL* baseline (0.5%), whereas **Round4_2020** tests large-scale disambiguation across a massive target set, where performance is measured strictly against available ground-truth annotations. The **SN** dataset, however, is specifically engineered to include “new content”, facts and entities explicitly missing from the reference Knowledge Graph, to evaluate a system’s ability to avoid false-positive links. By evaluating these datasets, we measure ReFinED’s calibration between entity disambiguation and *NIL* detection across varying levels of knowledge-graph completeness.

3.3.1 Computational Demands and Annotation Density

The selected datasets vary considerably in both scale and annotation characteristics, each posing distinct challenges for transformer-based inference. The magnitude of a dataset directly influences the computational behaviour of transformer-based models like ReFinED, which performs mention detection and disambiguation in a single forward pass rather than as a sequential feature pipeline. Large-scale datasets such as **2T_Round4** push the limits of sequence length and GPU memory, requiring careful selection of inference batch size, which is empirically determined in section 4.2.

Annotation density, the proportion of cells with ground-truth entity labels, varies significantly across the chosen datasets and presents a distinct additional challenge. In the **SN** dataset, a high proportion of *NIL* cases serve as a direct test of ReFinED’s internal confidence thresholds. Unlike supervised re-ranking systems that rely on secondary classifiers to filter predictions, ReFinED must rely solely on its normalised confidence score to abstain from linking ambiguous or out-of-vocabulary mentions. This evaluation therefore examines whether the model’s uncertainty signals remain reliable across datasets with varying label density and noise levels.

3.3.2 Naming Variability and Ambiguity

The way entities are named in tables is different from how they appear in regular text. In a normal sentence, the grammar and the surrounding words give the model clear clues to help it pick the right entity. In a table, however, a cell often contains nothing more than a short label, an abbreviation, or a technical code.

This creates a significant problem with ambiguity. For example, in the HardTables dataset, many different entities share the exact same name, such as several different cities called “Springfield”. Without a full sentence to explain which one is which, the model has to look elsewhere. Additionally, the Company, Movies, and SN datasets use specific jargon and non-standard abbreviations that might not be in the model’s original training data.

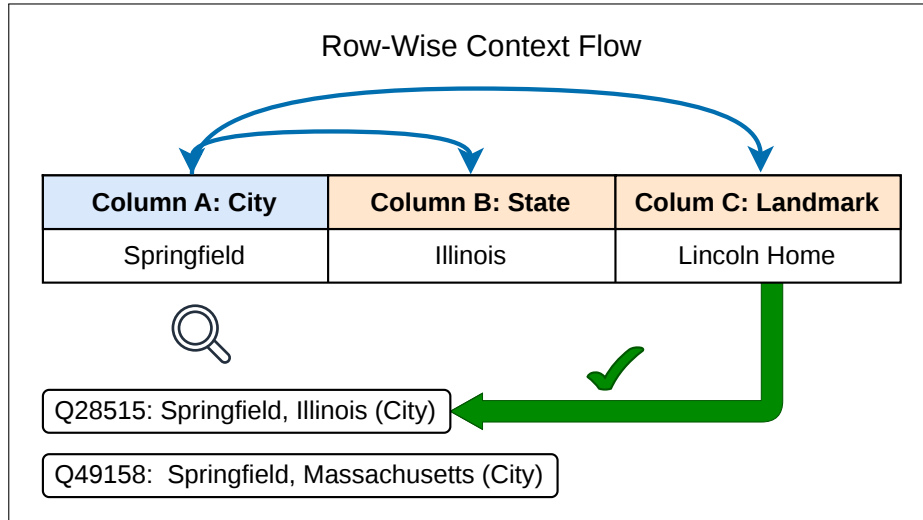


Figure 3.1: Row-wise context disambiguation: neighbouring cells (State and Landmark) resolve the ambiguous “Springfield” mention to the correct Wikidata QID.

As illustrated in Figure 3.1, systems may use row-wise contextual information. By incorporating context from neighbouring cells within the same row, such as “IL” (State) and “Lincoln Home” (Landmark), transformer models such as ReFinED can infer semantic relationships that are not available from the primary cell in isolation. This evaluation

therefore examines whether row-level contextual signals can compensate for the lack of traditional linguistic structure typically found in natural text.

3.4 Data Preparation and Preprocessing

Preparing the evaluation pipeline required addressing structural differences across dataset sources to ensure consistent input to ReFinED. This section describes the input formatting, ground-truth alignment, and dataset-specific handling applied across all evaluated corpora.

Input Formatting. All input tables are read CSV files without headers using `pandas`, with all values cast to strings. Cell values are accessed directly by zero-indexed row and column coordinates (r, c) . Three prediction modes were implemented. In *cell* mode, only the target cell value is passed to ReFinED as input. In *row-context* mode, the target cell together with all remaining values in the same row are concatenated into a single string using a pipe delimiter, forming the input sequence:

```
cell_text | col_1 | col_2 | ... | col_n
```

This provides ReFinED with row-level context during entity linking. In *label* mode, the column header is prepended to the cell value as `header: cell`, providing metadata without the full row context. This mode was introduced as a supplementary ablation and is evaluated as Configuration D in section 5.2. Out-of-bounds coordinates are silently skipped during sample construction.

Ground Truth Alignment and NIL Handling. Each dataset provides a target file listing the cell coordinates (r, c) to be evaluated, and a separate ground-truth file mapping coordinates to Wikidata QIDs. These are aligned via an inner join on `(table, row, col)`. Target cells with no matching ground-truth entry are excluded from the final evaluation sample. This ensures the performance metrics reflect the model’s accuracy on the annotated intersection of the dataset, rather than assuming unlabelled cells are implicit *NIL* entities. Some datasets also contain explicit *NIL* strings in the ground-truth file (e.g., the Spend Network dataset), which are handled equivalently. On the prediction side, if ReFinED returns no entity span for a given input, the prediction is also treated as *NIL*, and a *NIL* prediction against a *NIL* ground truth is counted as a true positive.

Dataset-Specific Handling. Minor structural inconsistencies were encountered across datasets. The `Round1_T2D` and `Round3_2019` targets file store coordinates in `(table, col, row)` order rather than the standard `(table, row, col)` order used by all other datasets and their corresponding ground-truth files. This was corrected by explicitly specifying column names during file loading. Additionally, the `Companies`, `Movies`, and

SN datasets include a header row in their table CSV files. Since all tables are read without a header, row indices from the ground truth are offset by one to account for this.

3.5 Evaluation Metrics and NIL-Awareness

While the mathematical foundations of Precision, Recall, and F1-score are established in chapter 2, their application to this research requires a specific operational definition of “correctness”, particularly regarding *NIL* entities.

Operational Definitions of Error Classes. Each model prediction is categorised based on its alignment with the ground-truth (*GT*) mapping. Let e_{pred} be the model’s prediction and E_{gold} be the set of valid identifiers.

- **True Positive (TP):** Recorded if the prediction matches ground-truth ($e_{pred} \in E_{gold}$). This includes cases where both the prediction and the ground truth are *NIL*.
- **False Negative (FN):** Recorded for any instance where a match is not achieved ($e_{pred} \notin E_{gold}$). This captures both omissions (predicting *NIL* when a link exists) and incorrect assignments.
- **False Positive (FP):** Recorded when the model predicts a specific entity ($e_{pred} \neq NIL$) that does not match the ground truth.

NIL-awareness. A core component of this evaluation is the model’s ability to abstain from linking. A correct *NIL* prediction, where the model returns no entity and the ground truth is also *NIL*, is explicitly counted as a True Positive. This is relevant for datasets with high open-world ratios, such as the SN dataset (22.9% *NIL*), and those where they serve as explicit adversarial targets, such as 2T_Round4 (0.5%). In datasets where the number of target cells exceeds the available ground-truth annotations (most notably Round4_2020), unannotated targets are excluded from the metric calculations. By performing evaluation strictly on the annotated intersection, we avoid unreliable assumption that unlabelled cells are implicit *NIL* cases, ensuring that the reported precision and recall reflect performance against verified labels only.

Global Score Aggregation. To ensure results reflect model performance at a corpus scale, all reported metrics use global aggregation. Rather than averaging scores per table (macro-averaging), *TP*, *FP*, and *FN* counts are accumulated into a single global pool for each dataset. This prevents small, high-accuracy tables from disproportionately influencing the final score and provides a more robust measure of performance on a large-scale relational data.

3.6 Limitations

Knowledge Graph Boundaries. ReFinED is zero-shot capable, meaning it can link to entities not seen during training by using entity description and fine-grained entity types from Wikidata [11]. However, performance is still bounded by the Wikidata snapshot used at inference time, entities that do not exist in Wikidata at all cannot be linked and will be predicted as *NIL*. A low recall in certain domains may therefore reflect gaps in the Knowledge Graph rather than a failure of the model’s linguistic capabilities.

Contextual and Computational Limits. The context mode concatenates all cell values in a row into a single pipe-delimited string, which is passed to ReFinED for entity linking. ReFinED internally tokenises this input using RoBERTa with a maximum sequence length of 512 tokens [11]. For tables with wide rows or long cell values, the effective context available for disambiguation may therefore be limited by this bound.

Chapter 4

Implementation and System Integration

This chapter describes the technical implementation of the ReFinED entity linking model, covering the full pipeline from experimental setup to system integration. It begins with the computational environment and hardware configurations used across the fine-tuning and benchmarking phases, followed by the model configuration and inference strategy used for evaluation. The automated evaluation pipeline developed to collect and log results across all datasets is then described, along with the back-end service built to expose ReFinED as a REST API. Finally, the integration with Koala-UI is presented, an open-source workspace for tabular data management and entity reconciliation into which ReFinED was connected as an additional reconciliation provider.

4.1 Computational Environment and Infrastructure

Understanding the hardware, software, and environment used is essential for reproducibility and contextualising runtime and efficiency observations reported later in this thesis. Describing the experimental setup also justifies configuration choices that affect performance, such as GPU selection, batch size limits, and memory constraints.

The experiments were conducted across two distinct systems. System A serving as the primary environment for API development described in section 4.4 and general testing and analysis, using an NVIDIA RTX 4070, while System B was used for both large-scale inference and benchmarking as well as fine-tuning experiments described in chapter 6, using an NVIDIA RTX 4070 Ti. Both systems ran Python 3.8.20 and PyTorch 2.4.1 within isolated virtual environments under the Windows Subsystem for Linux (WSL2) on Ubuntu, providing a Unix-like execution environment on Windows 11 host machines.

Dependency isolation via virtual environments ensured consistent and reproducible runs across both systems.

Table 4.1: Experimental Hardware Configurations

Component	System A	System B
CPU	Intel i5-13600K, 3.50 GHz	AMD Ryzen 5 3600, 3.60 GHz
RAM	32 GB DDR4 3600 MHz	32 GB DDR4 3200 MHz
GPU	RTX 4070, 12 GB VRAM	RTX 4070 Ti, 12 GB VRAM
Storage	HDD 2 TB	NVMe SSD 2 TB
OS	Windows 11 / WSL2 Ubuntu	Windows 11 / WSL2 Ubuntu
Python / PyTorch	3.8.20 / 2.4.1	3.8.20 / 2.4.1
Primary Role	API Development & Analysis	Fine-Tuning, Benchmarking & Evaluation

Both GPUs share the same 12 GB VRAM capacity, although the RTX 4070 Ti offers higher memory bandwidth and compute throughput, making it more suitable for the repeated large-scale inference runs required during benchmarking. All benchmark experiments were executed on GPU. Although CPU inference was verified as functionally equivalent, it was impractical for large-scale evaluation due to a significant reduction in throughput. The ReFinED model and its associated entity embeddings are loaded entirely into GPU memory at startup and kept resident throughout a run, meaning VRAM directly constrains the maximum feasible batch size.

Software Dependencies

The core ReFinED framework requires a standard set of deep learning and NLP libraries, including `torch`, `transformers`, and `nlTK`. For the experimental pipeline developed in this thesis, several additional libraries were introduced. `pandas` was used for dataset loading and preprocessing, providing a convenient interface for handling the CSV and JSON formats across the SemTab benchmarks. `matplotlib` was used to generate performance visualisations, with `scikit-learn` providing precision-recall curve computation and AUC scoring via its `precision_recall_curve` and `auc` utilities. The back-end service described in section 4.4 introduced `FastAPI`, `uvicorn`, and `pydantic` for REST API construction, request validation, and asynchronous job handling.

The initial environment setup proved to be a significant technical difficulty. While the ReFinED `requirements.txt` suggested a minimum version for the `transformers` library, testing revealed that newer versions introduced breaking changes to the model loading logic. Because of this, as well as Python version requirements, changes to the environment were made specifically for `transformers==4.2.2` and `python 3.8.20`. Resolving these

dependency issues required extensive troubleshooting and the use of virtual environments to ensure the RoBERTa-based weights and CUDA kernels remained compatible across both System A and System B. A full list of dependencies and version constraints is provided in `requirements.txt` on the project’s GitHub¹.

4.2 Model Configuration and Inference Strategy

Documenting how ReFinED was configured and executed ensures that the experiments are reproducible and that configuration choices are properly motivated. This section covers the selection of model variants, entity set, and prediction mode, as well as the inference strategy adopted for large-scale evaluation.

Model and Entity Set Selection. ReFinED provides several pre-trained model variants and entity set configurations. To determine the optimal combinations, a preliminary evaluation was conducted across four configurations on the `Companies`, `Movies`, and `SN` datasets, comparing the `wikipedia_model` and `wikipedia_model_with_numbers` variants against both the `wikipedia` and `wikidata` entity sets. The `wikipedia_model_with_numbers` variant extends the base model by incorporating SpaCy numerical entity types in the mention detection layer, including `DATE`, `CARDINAL`, `MONEY`, `PERCENT`, `TIME`, `ORDINAL`, and `QUANTITY`. This allows the model to correctly identify and bypass non-entity tokens that would otherwise introduce noise into the linking pipeline. The `wikidata` entity set resolves mentions against approximately 33 million filtered entities, compared to the approximately 6 million Wikipedia-linked entities in the `wikipedia` set.

The combination of `wikipedia_model_with_numbers` and the `wikidata` entity set consistently outperformed all other configurations. Switching from the Wikipedia entity set to Wikidata alone yielded the most significant gains, with the `Companies` dataset showing an improvement of approximately 20% in accuracy. Enabling numerical type detection further improved F1 scores across all datasets by reducing false positives from non-entity spans. Based on these findings, `wikipedia_model_with_numbers` with the `wikidata` entity set was used as the standard configuration for all subsequent experiments.

Prediction Mode. ReFinED performs entity linking at the mention level, processing text and returning linked entities for detected spans. Three inference strategies were evaluated in this thesis. In **cell-level** inference, each cell is passed to the model independently. In **row-context** inference, surrounding row cells are included in the input to provide additional context for disambiguation. In **label-mode** inference, the column header is prepended to the cell value, providing structural metadata without the computational overhead of full row context. Cell-level inference serves as the baseline due to its feasibility across all dataset scales. Row-context inference requires formatting entire

¹Python libraries used: <https://github.com/enRichMyData/ReFinED/blob/main/requirements.txt>

rows into longer input sequences, significantly increasing both inference time and VRAM consumption, making it infeasible for the largest datasets. Label-mode serves as a middle ground, offering improved disambiguation over cell-level inference at a lower computational cost than row-context mode. For these cases, row-context results are reported as unavailable in chapter 5, where the impact of all three inference strategies on disambiguation performance is examined and compared.

Batch Size Selection. To identify the optimal inference configuration, a preliminary sweep was conducted on the `Round1_T2D` dataset across batch sizes 8, 16, 32, 64, 128, and 256 in both cell-level and row-context prediction modes. Results showed that batch size had no meaningful effect on prediction quality, with F1 scores remaining stable across all tested values. Throughput increased with larger batch size in cell mode, with diminishing returns beyond batch size 32. Row-context mode proved significantly more memory intensive due to longer input sequences, with batch size 128 consuming 12.33 GB of VRAM, approaching the 12 GB hardware ceiling of the RTX 4070. Batch size 64 caused severe performance degradation in row mode, increasing inference time from approximately 4 seconds to over 12 minutes for 1,000 samples. Based on these findings, a batch size of 32 was explicitly selected for all primary experiments, as it represented the optimal trade-off between throughput and VRAM stability across both prediction modes.

Sequential Inference Profiling. Prior to the full benchmark run, a profiling run was done on sequential single-text inference using smaller datasets to characterise the computational behaviour of ReFinED. Using `cProfile`, the dominant cost was found to be the RoBERTa forward passes, accounting for the majority of cumulative runtime, with tensor preparation (`process_tensors`) and linear layer operations representing secondary costs. Python-level overhead from preprocessing and entity mention identification was minimal, confirming the major bottleneck to lie in transformer inference rather than surrounding pipeline logic.

A notable observation was the warmup effect: the first inference run over 100 texts took approximately 5.22 seconds, while subsequent warm runs averaged 1.38 seconds, a reduction of roughly $4\times$. This overhead is attributed to one-time costs including GPU memory transfer of model weights, CUDA kernel initialization, and tokeniser cache population. As a consequence, the model was kept loaded in memory across all dataset evaluations rather than reloaded between runs, avoiding repeated warmup overhead for the datasets after. These findings confirmed that sequential per-text inference significantly underutilises GPU parallelism, motivating the use of batched inference, where multiple texts are forwarded through the model simultaneously, was therefore adopted as the primary execution strategy.

4.3 Automated Evaluation Pipeline

To ensure consistent and reproducible evaluation across all datasets, a modular Python-based pipeline was developed alongside the core ReFinED framework. Rather than running experiments manually, the pipeline automates the full sequence from dataset loading and preprocessing to model inference, metric computation, and result logging. This section describes the structure of the pipeline, the role of each major script, and how results were collected and stored.

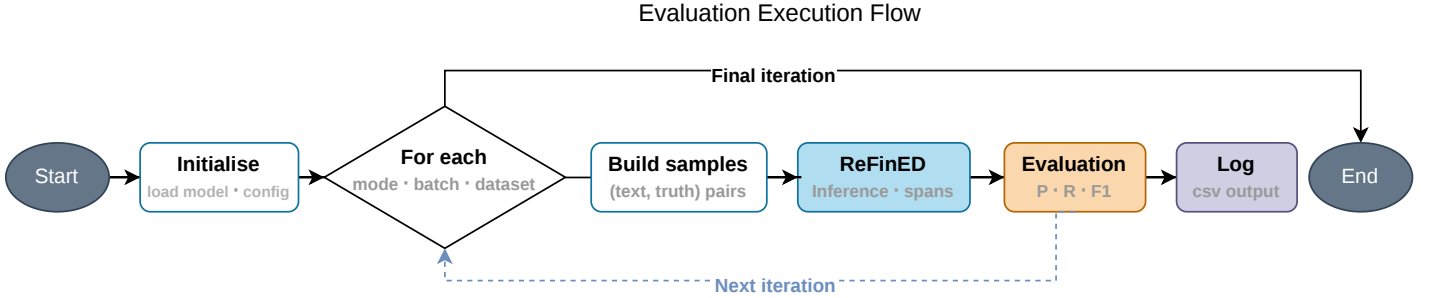


Figure 4.1: Execution flow of the evaluation pipeline, from dataset loading through inference to metric computation and result logging.

Pipeline Overview. The evaluation pipeline is organised around two primary scripts: `datasets.py`, which acts as the main runner responsible for loading, parsing, and preprocessing each dataset before passing data to the model, and `accuracy.py`, which handles all metric computation and result aggregation. Supporting utilities are provided by `test_utils.py`, which exposes the three core model interaction functions: `load_model`, `run_refined_single`, and `run_refined_batch`, used consistently across all scripts. Additional scripts including `benchmark.py` for profiling, `log_summary.py` for human-readable log analysis, and `pr_curve.py` for precision-recall visualisation complement the pipeline without being part of the main execution flow. All scripts are located in the `my_tests` directory of the project repository.

Dataset Loading and Preprocessing. `datasets.py` serves as the entry point for all benchmark runs. For each configured dataset, it resolves file paths, loads table data and ground truth annotations, and normalises entity identifiers to a consistent format for comparison. Some datasets vary slightly in their source format, some use CSV tables with separate ground truth files, others use JSON. `datasets.py` abstracts over these differences through dataset-specific parsing logic. Rows or cells that cannot be matched against a valid ground truth entry are skipped gracefully, ensuring robust handling of formatting inconsistencies across the different dataset formats. After preprocessing, the prepared inputs are passed to `test_utils.py`'s `run_refined_batch` function, which submits them to the loaded ReFinED model in configurable batches.

Metric Computation. The `accuracy.py` script computes precision, recall, and F1 scores by comparing predicted entities against ground truth QIDs at the cell level. Matches are classified as true positives, while mismatches or *NIL* predictions for existing gold annotations are treated as false positives and negatives, respectively. To support the throughput analysis in chapter 5, inference times are recorded concurrently with these accuracy metrics.

Result Logging. All results were written to a dated CSV log file on execution, with filenames encoding the model variant and run date for traceability. Four main log files were produced across the full set of experiments: `confidence_score_wiki_2026-05-08.csv`, containing per-prediction confidence scores for the stock Wikipedia model, `experimental_results_wiki_2026-05-08.csv`, containing aggregated precision, recall, and F1 scores per dataset for the stock model, `error_analysis.csv`, containing per-prediction records with input text, predicted and ground-truth QIDs, confidence scores, and correctness labels used to identify and categorise systematic failure patterns, and `experimental_results_f1_0.8972_2026-03-07.csv`, containing the equivalent results for the fine-tuned model. This naming convention allowed results from different model variants to be stored and compared without risk of overwriting. Log parsing and cross-run comparison were handled separately by the `log_summary.py` script.

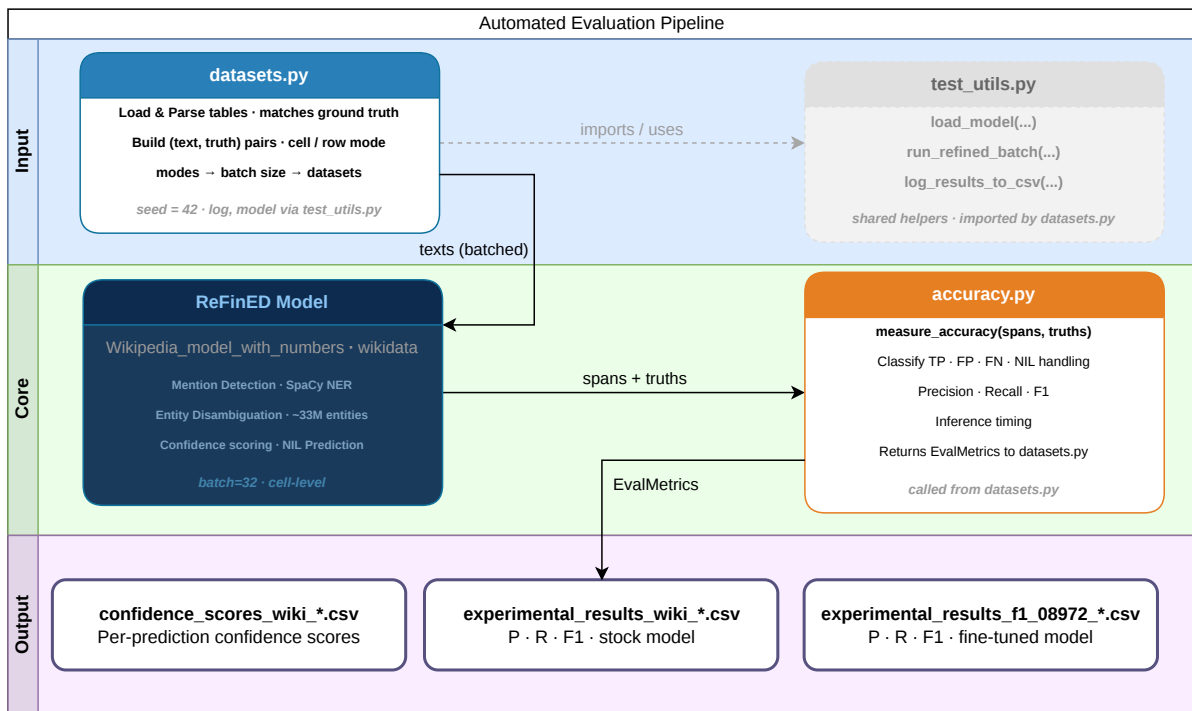


Figure 4.2: Automated evaluation pipeline organised across input, processing, and output layers, showing script roles and data flow

4.4 Back-end Service Architecture

To demonstrate how ReFinED could be integrated as a reconciliation provider within an existing annotation ecosystem, it was wrapped as a lightweight back-end service using FastAPI, following the same provider interface pattern used by Koala-UI for systems such as Crocodile² and LionLinker³. The service was developed as a prototype and technical demonstration rather than a production-ready deployment, reflecting the scope and time constraints of the thesis. All services were containerised using Docker, enabling straightforward local deployment and integration testing alongside Koala-UI, Crocodile, and LamAPI.

To enhance the portability of this service and simplify the management of dependency requirements identified in section 4.1, the back-end was containerised using Docker. As the development process involved a learning curve regarding containerisation, this phase focused on implementing a functional multi-stage build and ensuring GPU passthrough via the NVIDIA Container Toolkit. By using online documentation and AI-assisted debugging for configuration errors, a robust Dockerfile was produced. This ensures that the specific environment, including the fixed versions of Python and Transformers, can be deployed consistently on any host machine, abstracting away the underlying environment complexities.

4.4.1 REST API Design and Endpoints

The API is structured around two categories of endpoints: synchronous instant analysis and asynchronous job-based processing. All endpoints are protected by API key authentication via a custom `X-API-Key` header, implemented in `security.py` and applied globally to the router via FastAPI's dependency injection system.

The `POST /link` endpoint provides synchronous entity linking for a single input string, returning predicted entities, Wikidata QIDs, confidence scores, and coarse mention types immediately. This endpoint is used for real-time testing and lightweight queries rather than bulk table processing, and initially intended as a quick test of functionality and JSON formatting.

For tabular input, the service exposes a job-based workflow modelled after the Crocodile provider interface. A client submits a table via `POST /jobs`, specifying a header, rows, and a target column for linking. The service assigns a unique job identifier, stores the job in an in-memory store, and begins the linking task as a FastAPI background task. Two submission modes are supported: *inline*, where all rows are submitted in a single request, and *multipart*, where rows are streamed in parts via `POST /jobs/{job_id}/parts`

²<https://github.com/enRichMyData/crocodile>

³https://github.com/enRichMyData/lion_linker

and processing is triggered upon finalisation via `POST /jobs/{job_id}/finalize`. Job progress and status transitions, from `queued` through `running` to `done` or `failed`, are tracked and exposed via `GET /jobs/{job_id}`. Once completed, results are retrievable in paginated form via `GET /jobs/{job_id}/results`, returning linked entities formatted in the Koala-compatible JSON structure with candidate rankings, confidence scores, and column type metadata.

Rather than persisting job data to an external database, all job state is maintained in a module-level Python dictionary (`JOBS`), acting as in-memory store for the duration of the service’s lifetime. This design decision was made pragmatically: it avoided the overhead of configuring a database such as MongoDB, used by Crocodile, and was sufficient for the single-user, thesis-scoped testing environment in which the service was evaluated. It does, however, mean that all job data is lost upon service restart, and the design does not support concurrent multi-user workloads.

4.4.2 Model Serving and Memory Management

ReFinED is loaded once at service startup, when the API router module is first imported by FastAPI. The model is held inside memory for the full lifetime of the service process, avoiding the overhead of repeated initialisation across requests. Upon loading, the model occupies approximately 0.73 GB of allocated VRAM, with 0.78 GB reserved by the CUDA memory allocator on system A, the RTX 4070. This approach is well-suited to the single-user prototype context, where cold-start latency (the time required to download, load, and warm up the model) would otherwise dominate response time for short sessions. The service uses a named Docker volume to cache model weights between container restarts, meaning the full download from the ReFinED S3 storage is only required on first deployment

The service supports both CPU and GPU inference, configurable via the `MODEL_DEVICE` environment variable in the `.env` file. On CPU, a compatibility patch is applied to suppress unsupported autocast dtype warnings arising from ReFinED’s internal use of `torch.amp.autocast`. A `MockModel` data structure was also implemented for development and testing purposes, simulating model behaviour with artificial latency and random entity outputs, allowing the API and job pipeline to be exercised without loading the full model, a practical necessity given the model’s memory footprint on development hardware lacking a dedicated GPU.

Token limits are not explicitly enforced at the API layer. Each cell value is passed individually to ReFinED’s `process_text` method, which internally tokenises the input using RoBERTa’s tokeniser. Inputs exceeding RoBERTa’s 512 token context window are silently truncated at the tokeniser level. In practice this is unlikely to affect standard tabular cell

values, which are typically short strings, but presents a potential failure point for unusually verbose or concatenated row-context inputs.

A notable limitation of the current implementation concerns concurrency. The background task function `run_refined_task` is declared `async` but internally calls `run_refined_single`, which is a synchronous, blocking function. Under FastAPI’s default even loop behaviour, this means that if two jobs are submitted simultaneously, the second job’s inference will block until the first completes, as the synchronous call occupies the event loop. This was not a concern during development given the single-user testing context, but represents a clear path for improvement: wrapping inference calls in `asyncio.run_in_executor` or adopting a dedicated task queue would allow concurrent job processing in a wider-use deployment.

4.5 Integration with Koala-UI

Koala-UI is an open-source React and FastAPI workspace for tabular data management and entity reconciliation, developed and maintained at SINTEF. It supports multiple reconciliation providers simultaneously and offers candidate exploration, score-based sorting, column type classification, and enriched CSV export. The ReFinED back-end developed in this thesis was integrated into Koala-UI as an additional reconciliation provider, enabling entity linking results to be inspected interactively through its candidate exploration interface. The integration was validated both locally during development and through the SINTEF-hosted deployment where ReFinED is accessible alongside other providers such as Crocodile and LionLinker.

Connecting the ReFinED back-end to Koala-UI requires no code modification on either side. In the Koala-UI settings page, reconciliation providers are registered by supplying a base URL and an API key under the dedicated reconciliation services panel, which lists all available providers. Once the ReFinED service URL and token are entered, the provider appears as *ready* in the provider list alongside Crocodile, LionLinker, and the Wikidata reconciler, as shown in fig. 4.3.

4.5.1 Visualisation of Linking Results

The workflow from table upload to entity linking results proceeds through several steps in Koala-UI. After logging in, a user navigates to the datasets tab and creates a new dataset by providing a name and uploading a CSV table. Before reconciliation can be performed, columns must be classified as either named entity (NE) or literal (LIT) types. This classification can be performed manually via the column editor, or automatically using Koala-UI’s *moose* module, which samples table rows and uses a configured LLM service to assign NE and LIT subtypes, as shown in fig. 4.4.

Reconciliation services

Configure one reconciler at a time. Use search to quickly find a provider as the list grows.

Lion Linker

Entity linker powered by shared LLM + Lamapi retrieval.

ready

Crocodile

Asynchronous reconciler with candidate ranking.

ready

ReFinED

ReFinED entity linking API with background job processing.

ready

Wikidata Reconciler

Public OpenRefine-compatible Wikidata reconciliation endpoint.

ready

4 of 4 visible

ReFinED

ReFinED entity linking API with background job processing.

Leave blank to use the server default.

Key is stored for your profile.

Figure 4.3: Koala-UI settings page showing the reconciliation services panel with ReFinED registered as a provider alongside Crocodile, LionLinker, and the Wikidata reconciler.

Auto identify column types

Moose samples table rows and uses your LLM service to assign NE/LIT subtypes. You can tweak the result in the column editor afterward.

LLM provider

ollama

LLM model

gpt-oss:120b

Leave blank to use the server default.

LLM API key is stored for your profile.

UPDATE KEY

CANCEL

RUN AUTO IDENTIFY

Figure 4.4: Koala-UI automatic column type identification interface, where the moose module uses a configured LLM to assign NE and LIT subtypes to table columns prior to reconciliation.

Once column types are assigned, reconciliation is initiated by selecting a provider from the reconciler drop-down and pressing the run button. The user also specifies the scope of reconciliation, choosing between selected cells, selected rows, the current page, or the full dataset. When ReFinED is selected as the provider, the table is submitted to the ReFinED back-end as a job via `POST /jobs`, and processing proceeds asynchronously in the background as described in section 4.4.1.

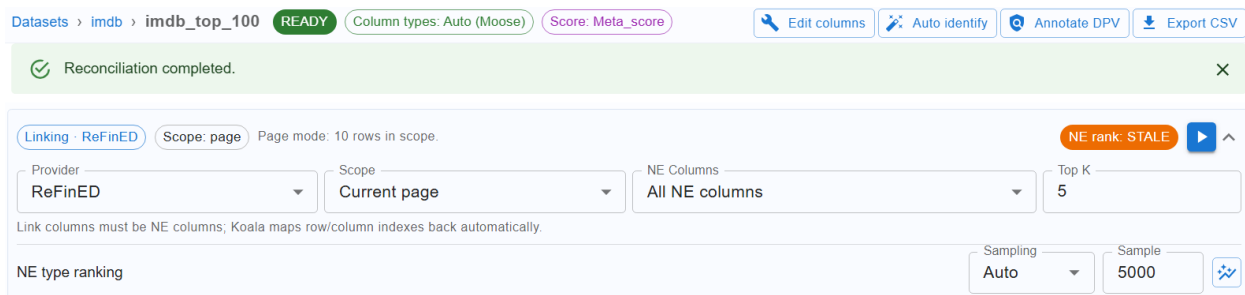


Figure 4.5: Koala-UI reconciler selection interface showing ReFinED selected as the active provider prior to running entity linking on the dataset.

Once the job completes, results are displayed directly in the table view. For each annotated cell, users can inspect the top- k candidate entities returned by ReFinED alongside their confidence scores, Wikidata QIDs, and entity type information. Candidates are ranked by score, with the top prediction highlighted. This interface provides a practical demonstration of how ReFinED’s predictions can be explored and validated in an interactive tabular environment, and served as the primary means of qualitative result inspection during integration testing.

Series_Title	Released_Year	Runtime (min)	Genre	IMDB_Rating	Overview	Meta_score	Director	Star1	No_of_Votes	Gross
NE: OTHER	NUMBER xsd:integer	NUMBER xsd:integer	STRING LIT:STRING	NUMBER xsd:decimal	STRING LIT:STRING	NUMBER xsd:decimal	NE: PERSON	NE: PERSON	NUMBER xsd:integer	STRING LIT:STRING
The Shawshank Redemption REF: The Shawshank Redemption	1994	142	Drama	9.3	Two imprisoned men bond over a number of years, finding solace and eventual redemption through acts of common decency.	80.0	Frank Darabont	Tim Robbins	2343110	28,341,469
The Godfather REF: The Godfather	1972	175	Crime, Drama	9.2	An organized crime dynasty's aging patriarch transfers control of his clandestine empire to his reluctant son.	100.0	Francis Ford Coppola	Marlon Brando	1620367	134,966,411
The Dark Knight REF: The Dark Knight	2008	152	Action, Crime, Drama	9.0	When the menace known as the Joker wreaks havoc and chaos on the people of Gotham, Batman must accept one of the	84.0	Christopher Nolan	Christian Bale	2303232	534,858,444

Figure 4.6: Koala-UI dataset view following entity linking with ReFinED, showing the top-3 results with candidate entity references per row.

Clicking an annotated cell opens the candidate exploration panel, where the top- k predictions are listed with their Wikidata QIDs, confidence scores, and entity types. The highest-ranked candidate is marked as the winner and selected by default, as shown in fig. 4.7.

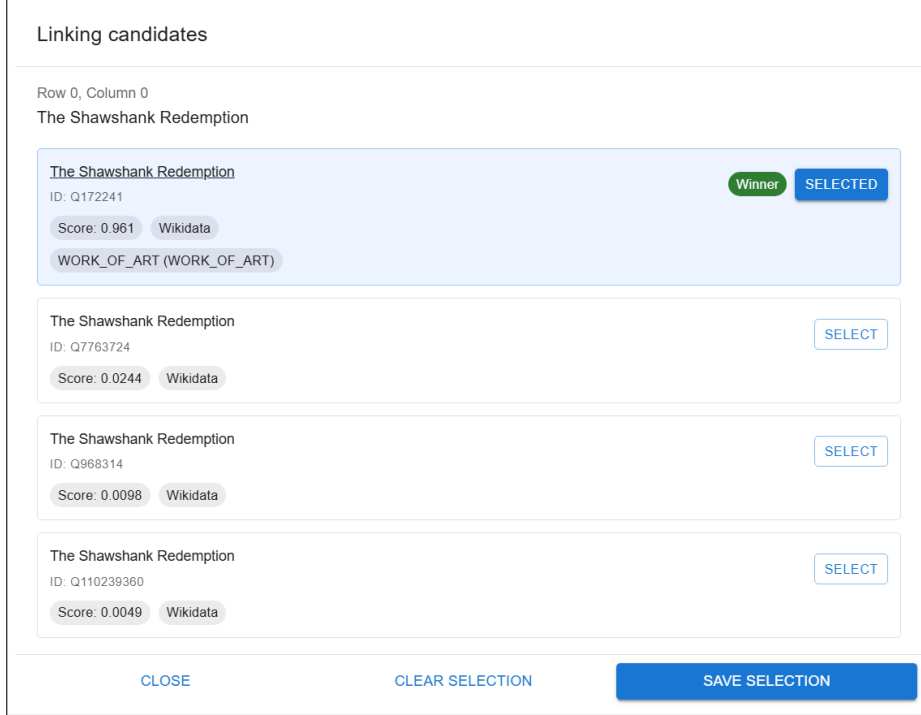


Figure 4.7: Koala-UI candidate exploration panel for *The Shawshank Redemption*, showing top- k candidates returned by ReFinED with confidence scores, Wikidata QIDs, entity types, and the top-ranked prediction highlighted as winner.

4.6 Practical Limitations

This section addresses implementation and deployment limitations specific to the back-end service and system integration, distinct from the dataset-level constraints discussed in section 3.6.

Row-Context Scalability. Row-context mode proved computationally infeasible for the three largest datasets in the benchmark: `Round3_2019` (406,827 samples), `Round4_2020` (994,921 samples), and `2T_Round4` (667,244 samples). Due to the significantly longer input sequences produced by row-context serialisation as well as the limited hardware of consumer-grade GPU hardware, inference times far exceeded practical limits, with `Round3_2019` alone requiring over ten hours without completing. These datasets were therefore evaluated in cell mode only, and row mode results are reported as unavailable for these three datasets in the results tables in chapter 5.

Knowledge Graph Staleness. ReFinED was trained on a fixed Wikidata snapshot, meaning entities created, merged, or renamed after the model’s knowledge cut-off cannot be correctly linked. This is particularly relevant for dynamic domains such as companies and political entities, where Wikidata evolves continuously. Ground truth files used

in this thesis are similarly snapshot-bound, meaning that QID redirects or deletions introduced after the snapshot date may silently affect evaluation correctness without any visible error.

NIL Prediction Conservatism. Based on the benchmark results, ReFinED shows a consistent tendency to predict *NIL* rather than commit to a low-confidence entity link. While this conservative behaviour improves precision, it consistently suppresses recall across all evaluated datasets. This pattern is reflected in the results throughout chapter 5, where precision consistently exceeds recall, and should be considered when interpreting the precision-recall trade-off discussed in section 5.3.

Single-Row Inference in the API. The back-end service processes each table row individually using `run_refined_single`, submitting one mention at a time to the model rather than exploiting ReFinED’s batched inference capability via `run_refined_batch`. This was a deliberate simplification for the prototype, prioritising implementation clarity over throughput. For large tables, this approach would be significantly slower than the batched evaluation pipeline used in the benchmark experiments, and replacing the per-row call with batched processing represents a straightforward improvement for future iterations of the service.

Service Deployment Constraints. When deployed as a persistent back-end service, ReFinED’s model weights occupy approximately 0.78 GB of reserved VRAM on startup, as measured on the RTX 4070 used during API development. While this footprint is relatively modest for a transformer-based model, limited VRAM presents challenges if inference is combined with other GPU workloads or if the service is run on hardware with less available memory. Following development, the open-source back-end was adopted and deployed by the Koala-UI maintainers at SINTEF on server-grade hardware, where these constraints are less pronounced. Additionally, job state is held in memory rather than persisted to a database, meaning all results are lost upon service restart, a known limitation of the prototype design discussed further in section 4.4.1.

Chapter 5

Evaluation

Evaluating a general-purpose zero-shot entity linking model such as ReFinED against systems purpose-built for specific benchmarks is inherently an uneven comparison. Yet this unevenness also makes the results informative: where ReFinED performs competitively despite having no dataset-specific training, it demonstrates genuine zero-shot capability, and where it falls short, the gap quantifies the cost of generalisation over specialisation. The SemTab challenge has historically been dominated by systems that are trained, tuned, and indexed specifically for its datasets, with top-10 average CEA F1 scores of 0.92 on Round4_2020 and 0.54 on the adversarial 2T_Round4 [25], and 0.91 and 0.90 on the synthesised **HardTablesR2** and **HardTablesR3** respectively [28]. ReFinED, by contrast, enters this evaluation without dataset-specific training, relying entirely on its pre-trained transformer backbone and Wikidata entity set. The question is not whether a specialised system outperforms a generalist, but whether ReFinED’s zero-shot neural approach can offer competitive accuracy, and under what conditions it succeeds or fails.

This chapter addresses that question across three areas. First, it benchmarks ReFinED’s CEA performance across all 11 evaluated datasets across cell-only, row-context, and column-header-label inference modes, establishing a performance profile relative to the SemTab competitive landscape. Second, it examines the computational trade-offs of large-scale transformer inference, analysing how inference strategy and prediction mode affect throughput and memory consumption in practice. Third, it presents an ablation study comparing the stock pre-trained model against the fine-tuned variant described in chapter 6, isolating the contribution of domain adaptation to overall performance. Together, these analyses provide the empirical foundation for the broader discussion in chapter 7.

5.1 Experimental Design

All experiments follow a standardised operation flow divided into four distinct phases, designed to manage the computational demands of the ReFinED transformer and maintain a verifiable audit trail of model predictions.

Before the benchmark loop begins, the ReFinED model and its associated weights are loaded once into GPU VRAM and kept there throughout all dataset evaluations. This avoids repeated cold start overhead, a deliberate design choice motivated by the profiling findings described in section 4.2, where the first inference run incurred a one-time warmup cost of approximately $4\times$ the steady-state inference time due to CUDA kernel initialisation and tokeniser cache population. Once initialised, tables are processed sequentially, while the actual inference is performed in batches. Following the configuration strategy established in section 4.2, all experiments were conducted using a fixed batch size of 32. This configuration was chosen to ensure peak throughput while maintaining VRAM stability across both prediction modes, avoiding performance degradation observed at higher batch sizes.

After each dataset completes, all relevant performance metrics are written to a structured CSV log. Each entry records the dataset name, prediction mode, model, batch size, sample counts, TP/FP/FN counts, precision, recall, F1, peak VRAM consumption, throughput, and total inference time. The raw data files demonstrating this consistency, including Wikidata-specific evaluation logs and per-dataset performance summaries, are documented in Appendix A. For precision-recall curve analysis, per-sample confidence scores and correctness labels are additionally written to a separate confidence log, enabling threshold analysis to be performed offline and independently of the main benchmark run. Final performance metrics are calculated using a global aggregation strategy rather than macro-averaging across tables. True Positives (TP), False Positives (FP), and False Negatives (FN) are summed across the entire corpus of a dataset before any ratios are calculated. By applying the operational definitions established in section 3.5 to this accumulated data, the resulting metrics provide a robust measure of performance at a corpus scale. This approach prevents small tables from disproportionately skewing the results of larger, more complex datasets.

5.1.1 Comparison Methodology

ReFinED is not evaluated in isolation. Its results are contextualised against a range of reference points drawn from the STI literature. As noted in section 2.5.1, the systems ReFinED is compared against were purpose-built, which creates an inherently unequal but informative basis for comparison.

All comparison results are sourced directly from their respective publications rather than

re-run locally, as reproducing the full suite of SemTab systems was outside the scope of this thesis. As a baseline, the top-10 average CEA F1 scores reported in the SemTab 2020 and 2021 results papers [25] [28] are used to contextualise ReFinED’s performance on Round4_2020, 2T_Round4, HardTablesR2, and HardTablesR3. For a more direct system-level comparison, LinkingPark [15], a top-3 SemTab 2020 participant, provides CEA F1 reference scores of 0.985 on Round4_2020 and 0.810 on 2T_Round4, representing a strong named-system baseline on those two datasets.

A qualitative comparison is drawn from Belotti et al. [17], which evaluates Alligator, DAGOBAN, TURL, TableLlama, GPT-4o, and GPT-4o-mini on overlapping datasets. However, a direct numerical comparison is complicated by the fact that several major models they investigate, such as TURL and TableLlama, are explicitly noted to lack *NIL* awareness (see Belotti et al., Table 2). These models effectively operate under a close-world assumption where they must select the most relevant entity from a provided candidate set. ReFinED, however, is evaluated under an open-world assumption where *NIL* predictions are valid and required when the correct entity is absent. This difference in how models handle or ignore out-of-knowledge entities makes their accuracy scores numerically distinct from ReFinED’s open-world F1 metrics.

All ReFinED experiments are evaluated on the hardware described in section 4.1, using the ground truth files described in chapter 3, ensuring that predictions are assessed on identical target cells mapped to the same Wikidata snapshot across all runs.

It should also be noted that the primary evaluation inputs consisted solely of cell values and their surrounding row context, without accompanying column headers or type labels. The contribution of column header context is addressed directly as Configuration D in section 5.2.

5.1.2 Experimental Hypotheses

The three research questions defined in section 1.3 motivate three corresponding hypotheses that guides the experimental design of this chapter. Each hypothesis is evaluated through the benchmark, ablation, and efficiency analyses presented in the following sections.

- **Benchmark Hypothesis (RQ1):** ReFinED’s transformer-based architecture will demonstrate competitive performance on tabular CEA benchmarks relative to existing STI approaches, and row-level contextual information will improve disambiguation over cell-only inference, particularly on datasets with high ambiguity such as HardTablesR2 and HardTablesR3. This is motivated by ReFinED’s zero-shot capability and RoBERTa’s context-sensitive embeddings described in section 2.4, which

should provide robustness to lexical noise without dataset-specific training.

- **Adaptation Hypothesis (RQ2):** Supervised fine-tuning on domain-specific tabular data will improve ReFinED’s entity linking performance over the general-purpose pre-trained model on in-domain datasets, with some degradation expected on out-of-domain datasets. This is consistent with prior work showing that domain-adaptive pre-training improves downstream performance when the target domain diverges significantly from the pre-trained distribution [30], as is the case when moving from Wikipedia prose to structured tabular data.
- **Efficiency Hypothesis (RQ3):** Row-context inference will bring measurable throughput and memory costs relative to cell-only inference, and batch size will present diminishing returns beyond a threshold that reflects the VRAM ceiling of consumer-grade hardware. This follows directly from the profiling observations in section 4.2, where row-context serialisation was shown to significantly increase input sequence length and VRAM consumption.

5.2 Ablation Study

To isolate the contribution of individual system components, four ablation configurations were evaluated across the benchmark suite. Configuration A evaluates the model using only the target cell string, establishing a baseline that measures the model’s internal lookup strength without any surrounding context, hence the cell-only inference label. Configuration B extends the input to include all neighbouring cells in the same row, as described in section 3.4, measuring the contribution of row-level context to disambiguation performance. Configuration C compares the stock `wikipedia_model_with_numbers` against the fine-tuned `merged_full` variant, isolating the impact of domain-specific adaptation on both in-domain and out-of-domain datasets. Configuration D evaluates the effect of prepending column header labels to cell values, measuring the contribution of structural metadata to disambiguation accuracy. The fine-tuning process is described in detail in chapter 6, while results are included here for direct comparison against the stock model baseline.

Configurations A and B: Impact of Row-level Context. Table 5.1 presents the cell-only and row-context F1 scores for all datasets where both modes were evaluated, along with the contextual gain (Δ F1). Bold values indicate the higher F1 score for each dataset, with a positive Δ F1 indicating row-context outperforms cell-only, and negative indicating the opposite. `Round3_2019`, `Round4_2020`, and `2T_Round4` are excluded from row-context evaluation due to computational infeasibility, as discussed in section 4.6.

Table 5.1: Ablation A vs B: Cell-only vs Row-context F1 (Stock Model)

Dataset	Cell F1	Row F1	Δ F1
movies	0.5779	0.7564	+0.1785
Round1_T2D	0.7047	0.7975	+0.0928
companies	0.8022	0.8416	+0.0394
SN	0.5482	0.5658	+0.0176
HTR1	0.5451	0.5272	-0.0179
HTR2	0.3825	0.3634	-0.0191
HardTablesR2	0.5733	0.5610	-0.0123
HardTablesR3	0.3578	0.3367	-0.0211
Round3_2019	0.5859	—	—
Round4_2020	0.6133	—	—
2T_Round4	0.6130	—	—

Note: Results are from the stock `wikipedia_model_with_numbers` model with the `wikidata` entity set.

The results reveal a clear split across dataset types. Row-context inference consistently improves performance on structured, semantically rich datasets, most notably **movies** (+0.1785), where film titles are frequently ambiguous without surrounding metadata such as director, year, or genre. **Round1_T2D** also benefits substantially (+0.0928), suggesting that well-formed web tables with informative column relationships provide useful contextual signals for disambiguation. The gains on **companies** (+0.0394) and **SN** (+0.0176) are more modest, reflecting the shorter and more abbreviated nature of corporate and procurement data where neighbouring cells add less semantic value.

In contrast, row-context mode consistently hurts performance on the adversarial HardTables and HTR datasets, with degradations ranging from -0.0123 to -0.0211 . These datasets are characterised by high lexical noise, synthetic typos, and high-ambiguity homonyms, conditions under which surrounding cells may introduce misleading context rather than helpful disambiguation signals. This phenomenon, where noisy neighbouring cells distort the model’s entity prediction, is examined further as a failure mode in section 5.4.

Configuration C: Fine-tuned vs. Stock Model. Table 5.2 presents the F1 score comparison between the stock Wikipedia model and the fine-tuned `merged_full` model across the four ablation datasets.

Table 5.2: Ablation C: Stock vs. Fine-tuned Model (F1 Score)

Dataset	Mode	Stock F1	Fine-tuned F1	Δ
companies	cell	0.8022	0.9172	+0.1150
movies	cell	0.5779	0.7465	+0.1686
SN	cell	0.5482	0.4952	-0.0530
Round1_T2D	cell	0.7047	0.7109	+0.0062
companies	row	0.8416	0.9394	+0.0978
movies	row	0.7564	0.9476	+0.1912
SN	row	0.5658	0.5110	-0.0548
Round1_T2D	row	0.7975	0.6073	-0.1902

Note: SN and Round1_T2D were not included in the fine-tuning corpus and serve as out-of-domain generalisation tests. Fine-tuning results are from the merged_full f1_0.8972 model (merged_full).

Fine-tuning yields substantial gains on the in-domain datasets, with F1 improvements of up to +0.1912 on **movies** in row-context mode and +0.1150 on **companies** in cell mode. The fine-tuned model also achieves a notably higher precision across both in-domain datasets, 0.9945 on **movies** (row) and 0.9922 on **companies** (row), suggesting that domain adaptation sharpens the model’s entity prior toward the target distribution.

The out-of-domain results present a more mixed picture of model robustness. Performance on SN drops in both modes (−0.0530 cell, −0.0548 row), while Round1_T2D shows little cell mode improvement (+0.0062) alongside a bigger row mode degradation (−0.1902). This pattern is consistent with what is known as catastrophic forgetting [31], a well-documented phenomenon in neural training where fine-tuning on a narrow domain causes the model to overwrite general-purpose weights, degrading performance on out-of-distribution data. The particularly severe row mode degradation on Round1_T2D suggests that the model’s attention mechanism has over-specialised to the specific horizontal context patterns found in **movies** and **companies**. While the cell mode still benefits from the ReFinED entity embeddings, the row mode attempts to apply specialised contextual logic to the general-purpose web tables of T2D, resulting in significant loss of generalisation. These trade-offs are discussed further in chapter 6 and chapter 7.

Configuration D: Impact of Column Header Context. Table 5.3 presents the F1 scores for cell-only and column-header-label inference modes across all 11 datasets. In label mode, the column header is prepended to the cell value as **header: cell**, providing the model with structural metadata without the full row context of configuration B.

Table 5.3: Ablation D: Cell-only vs. Column Header Label F1 (Stock Model)

Dataset	Cell F1	Label F1	Δ F1
Round1_T2D	0.7047	0.7775	+0.0728
Round3_2019	0.5859	0.6512	+0.0653
Round4_2020	0.6133	0.6281	+0.0148
2T_Round4	0.6130	0.6123	-0.0007
HTR1	0.5451	0.5746	+0.0295
HTR2	0.3825	0.3937	+0.0112
HardTablesR2	0.5733	0.5937	+0.0204
HardTablesR3	0.3578	0.3293	-0.0285
companies	0.8022	0.8511	+0.0489
movies	0.5779	0.6921	+0.1142
SN	0.5482	0.5908	+0.0426

Note: Results are from the stock `wikipedia_model_with_numbers` model with the `wikidata` entity set. Label mode prepends the column header to the cell value as `header: cell`.

Column header context improves F1 score on 9 out of 11 datasets, with the most significant gains observed on `movies` (+0.1142) and `Round1_T2D` (+0.0728). These datasets are characterised by semantically rich headers (e.g., “Director” or “Country”) which provide a strong prior for disambiguation. In contrast, gains are marginal on the HTR and HardTables datasets (+0.0112 to +0.0204), where headers often consist of generic placeholders (`col0`, `col1`) that offer no additional disambiguation context beyond the cell value itself.

Notably, as shown in the summary analysis (table 5.4), label mode achieves the highest average F1 score (0.6086) across all tested configurations, even outperforming the more computationally expensive row mode (0.5937). Given that label mode maintains a throughput of 209.97 txt/s compared to row mode’s 120.14 txt/s, it represents the most efficient trade-off between disambiguation accuracy and computational overhead.

The degradation on `HardTablesR3` (−0.0285) and the flat performance on `2T_Round4` (−0.0007) remain consistent with the adversarial nature of these datasets, where surrounding context, including headers, is frequently designed to be noisy or non-representative of the target’s cell entity type.

Table 5.4: System Performance Summary: Efficiency vs. Accuracy

Configuration	Mean F1 Score	Throughput (texts/s)	Peak VRAM (GB)
Cell-only	0.5731	238.26	1.91
Column Label	0.6086	209.97	1.87
Row-context	0.5937	120.14	3.51

Note: F1 scores are averaged across all 11 benchmark datasets, and were verified to be hardware-independent.

5.3 Results and Comparative Analysis

Across all 11 evaluated datasets, ReFinED demonstrates a consistent precision-recall imbalance, achieving higher precision at the cost of recall through its conservative *NIL*-prediction strategy. The following sections examine this pattern in detail, alongside the computational trade-offs between inference modes and a comparative analysis against existing STI approaches. Unless otherwise stated, all results are reported for the `stock_wikipedia_model_with_numbers` model with the `wikidata` entity set.

5.3.1 Precision-Recall Dynamics

A consistent pattern emerges across all evaluated datasets: ReFinED achieves substantially higher precision than recall, reflecting a conservative linking strategy in which the model abstains from predicting an entity when confidence is low, defaulting to *NIL* instead. This behaviour is a direct consequence of ReFinED’s internal scoring mechanism, which combines entity typing, description similarity, and entity priors into a final confidence score ω . A special *NIL* candidate with a fixed score of zero is included alongside all real entity candidates. *NIL* is predicted only when all candidate ω scores are negative, causing the model to abstain from linking [11]. While this improves precision, it systematically suppresses recall, particularly on datasets with high lexical noise or ambiguous mentions where the model’s uncertainty is highest.

Figures 5.1 and 5.3 present the Precision-Recall (PR) curves and F1 score against confidence threshold for the specialised and SemTab datasets in cell mode, whereas Figures 5.2 and 5.4 show the equivalent plots for row mode. Note that `Round3_2019`, `Round4_2020`, and `2T_Round4` are absent from the row mode figure as these datasets were evaluated in cell mode only, as discussed in section 4.6. Several observations stand out.

Specialised. On the specialised datasets, `companies` and `movies` show well-formed PR curves with AUC scores of 0.650 and 0.623 in cell mode, improving to 0.690 and 0.744 in row mode. A notable feature of the F1 vs. threshold plots is the broad plateau across con-

fidence values roughly between 0.1 and 0.9, indicating that the model’s default threshold is near-optimal for these datasets and that performance is relatively insensitive to threshold tuning. The *SN* dataset’s PR curve starts at precision 1.0 at near-zero recall before dropping sharply, reflecting the high *NIL* ratio (22.9%) in this dataset. The model achieves near-perfect precision only by linking very few entities, while F1 plateaus around 0.59 regardless of threshold, suggesting that the difficulty stems from the nature of the data rather than poorly calibrated confidence scores.

SemTab. On the SemTab datasets, *Round1_T2D* clearly outperforms all others with an AUC of 0.735 in cell mode and a peak F1 of 0.86, consistent with its status as the most well-structured dataset in the benchmark. *HardTablesR3* is the weakest performer with an AUC of only 0.284 and a peak F1 of 0.53, reflecting the scale and adversarial complexity of this dataset. The optimal confidence threshold for SemTab datasets appears around 0.3–0.4, beyond which F1 degrades as the model becomes increasingly conservative. This suggests that for high-volume SemTab-style annotation tasks, a slightly lower confidence threshold than the model default may be preferable to recover recall without substantially sacrificing precision.

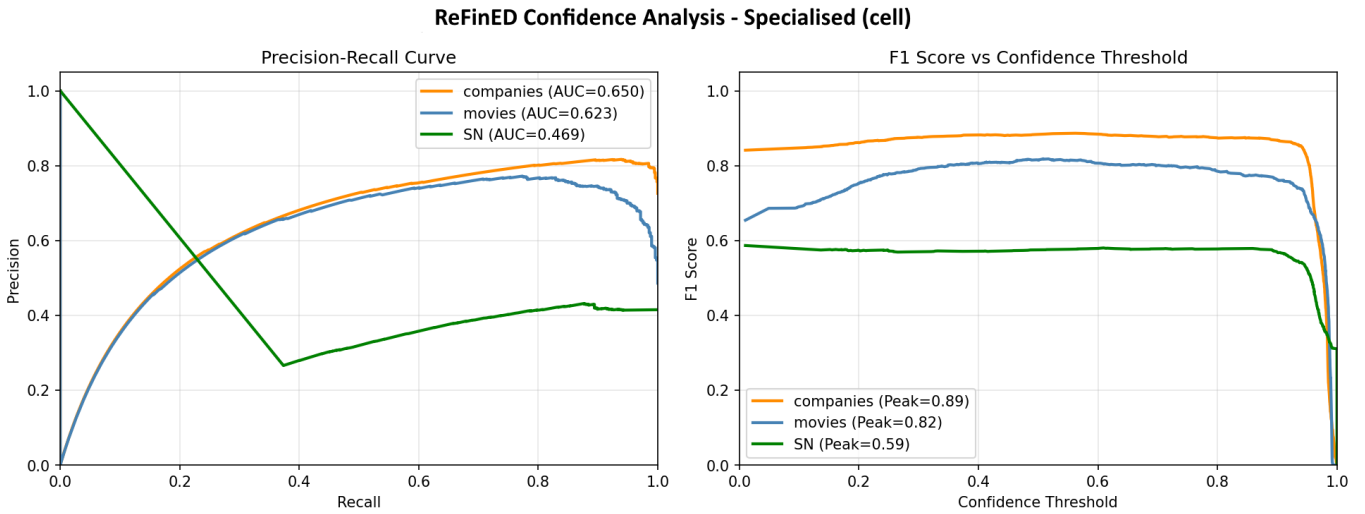


Figure 5.1: PR curves and F1 vs. confidence threshold, specialised datasets, cell mode. *SN* (AUC=0.469) is notably weaker than *companies* (AUC=0.650) and *movies* (AUC=0.623), reflecting its high *NIL* ratio.

ReFinED Confidence Analysis - Specialised (row)

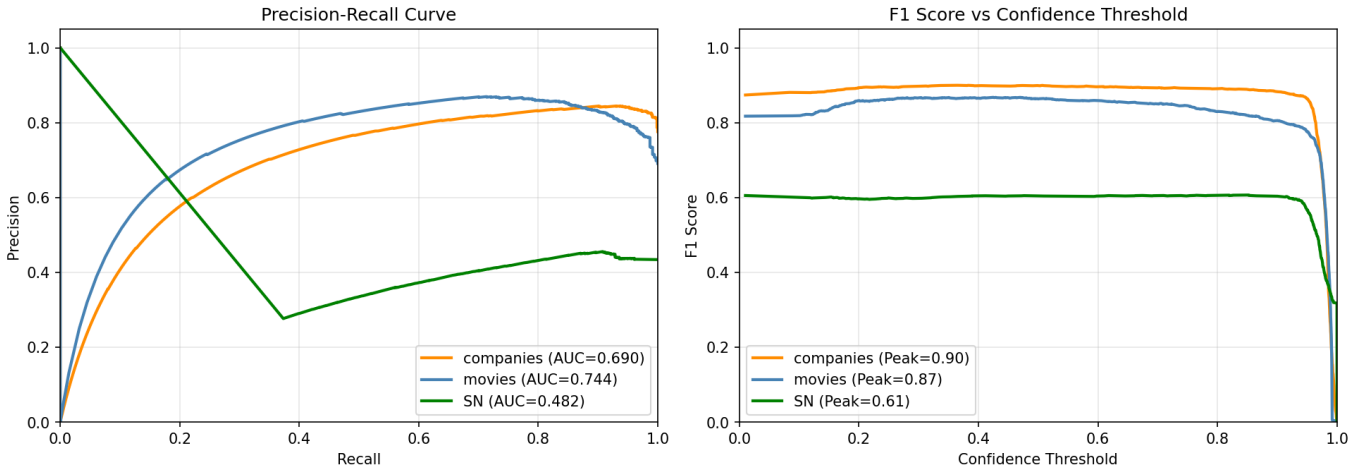


Figure 5.2: PR curves and F1 vs. confidence threshold, specialised datasets, row mode. **companies** (AUC=0.690) and **movies** (AUC=0.744) improve over cell mode; **SN** (AUC=0.482) remains constrained by its *NIL* ratio.

ReFinED Confidence Analysis - SemTab Rounds (cell)

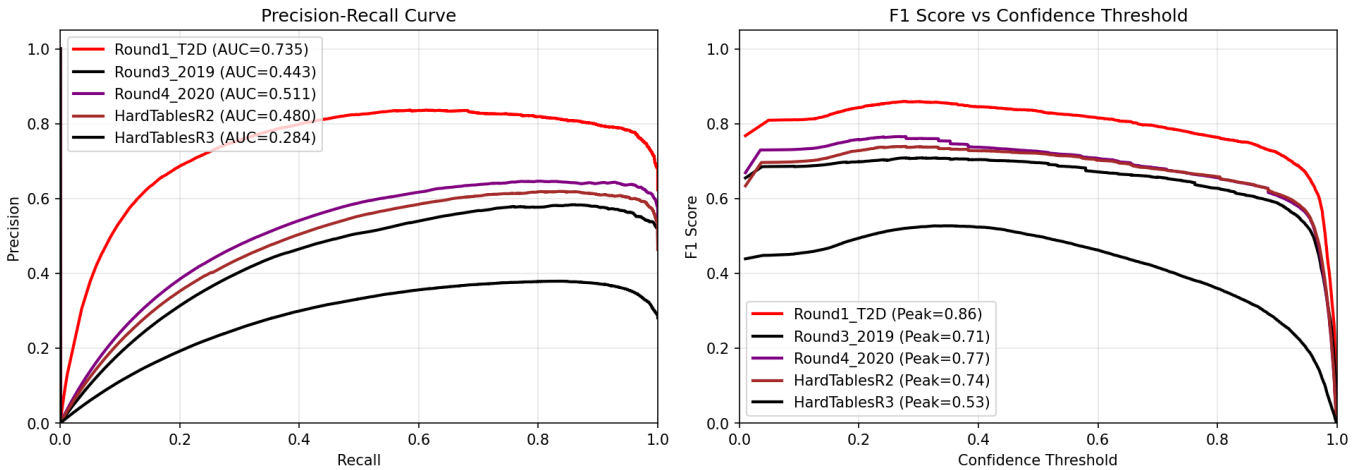


Figure 5.3: PR curves and F1 vs. confidence threshold, SemTab datasets, cell mode. **Round1_T2D** (AUC=0.735) leads, while **HardTablesR3** (AUC=0.284) reflects adversarial complexity.

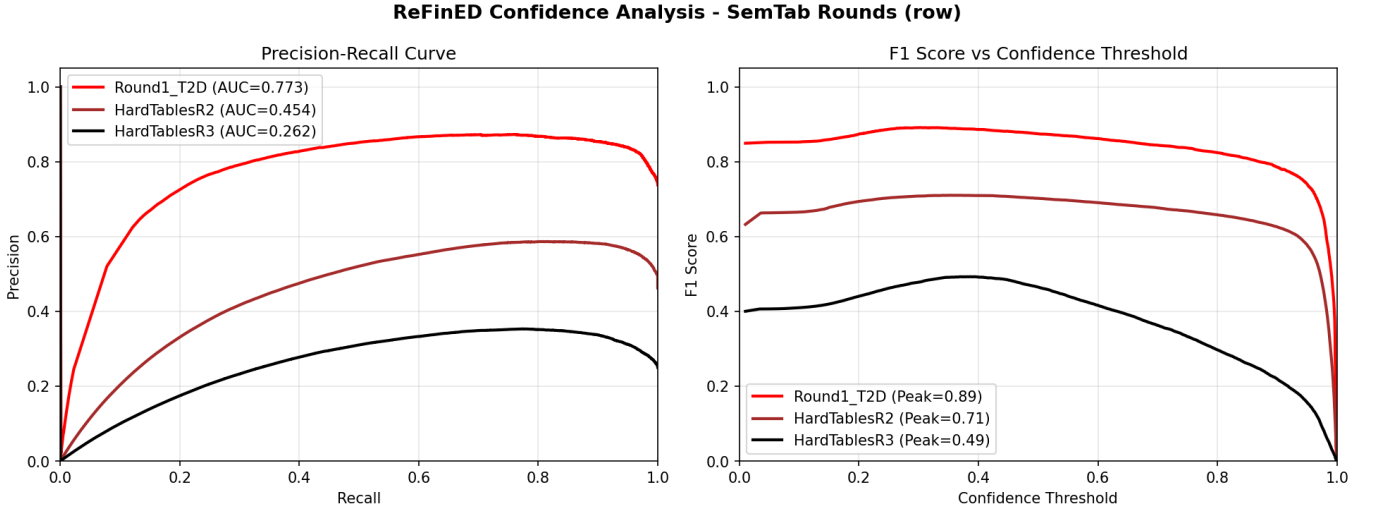


Figure 5.4: PR curves and F1 vs. confidence threshold, SemTab datasets, row mode. Round1_T2D (AUC=0.773) improves over cell mode; HardTablesR3 (AUC=0.262) degrades slightly, consistent with adversarial noise sensitivity.

5.3.2 Efficiency vs. Accuracy Trade-offs

A key practical consideration in deploying ReFinED for large-scale entity linking is the computational cost associated with different levels of structural context. Table 5.5 summarises the throughput and peak VRAM consumption across the cell-only, column label, and row-context modes, derived from the logged benchmark runs.

Table 5.5: Throughput and VRAM by Inference Mode (Stock Model on System B)

Dataset	Throughput (txt/s)			Peak VRAM (GB)		
	Cell	Label	Row	Cell	Label	Row
companies	117.1	120.7	28.1	1.43	1.41	5.56
movies	97.5	95.9	53.5	1.49	1.65	3.62
SN	198.0	205.7	69.4	1.45	1.29	2.85
Round1_T2D	210.7	176.5	91.3	1.69	1.68	7.46
HTR1	230.8	152.0	154.6	1.53	1.63	1.65
HTR2	236.2	156.5	144.1	1.56	1.68	1.92
HardTablesR2	309.3	224.1	192.1	1.90	2.03	3.25
HardTablesR3	261.9	171.9	228.2	1.82	1.70	1.75
Round3_2019	357.5	301.9	—	2.40	2.40	—
Round4_2020	227.3	356.9	—	3.16	2.68	—
2T_Round4	374.7	347.8	—	2.55	2.40	—
Average	238.3	210.0	120.1	1.91	1.87	3.51

Note: Empty cells indicate row-context mode was not evaluated due to computational infeasibility at scale. VRAM values reflect peak consumption during inference.

Switching from cell-only to row-context inference roughly halves average throughput, from 238.3 to 120.1 texts per second, while nearly doubling average peak VRAM consumption from 1.91 to 3.51 GB. Interestingly, the column label mode offers a performance middle ground, maintaining a high throughput of 210.0 txt/s while actually showing the lowest average peak VRAM at 1.87 GB. The impact of row-context is most notable on **companies**, where throughput drops from 117.1 to 28.1 txt/s, a reduction of approximately 4.2 $\times$ due to longer concatenated row strings produced by wide tables with many non-empty columns. In contrast, **HardTablesR3** shows a similar throughput trend to the other datasets in row mode compared to its label mode, but it remains the most efficient dataset for row-context processing (228.2 txt/s), likely because its short, sparse rows produce minimal overhead when concatenated.

The three largest datasets, **Round3_2019**, **Round4_2020**, and **2T_Round4**, were evaluated in cell and label mode only, as row-context inference proved computationally difficult at their scale on consumer-grade hardware. For **Round3_2019** alone, projected row-context runtime exceeded ten hours without completing. These observations directly informed the asynchronous job-based architecture of the Koala-UI back-end described in section 4.4, where cell mode is used as the default inference strategy. Row-context mode was not

exposed in the back-end service due to its computational demands, while label mode represents a viable alternative for users seeking improved disambiguation without significant performance degradation.

5.3.3 Comparative Analysis

Table 5.6 places ReFinED’s CEA F1 scores in the context against three external reference points on overlapping datasets: Alligator [12], a supervised re-ranking system whose benchmark datasets form the primary evaluation scope of this thesis; LinkingPark [15], a top-3 SemTab 2020 participant; and the top-10 average CEA F1 reported in the SemTab 2020 and 2021 results papers [25] [28].

Table 5.6: CEA F1 Comparison: ReFinED vs. STI Reference Systems

Dataset	ReFinED (cell)	ReFinED (row)	Alligator	LinkingPark	Top-10 Avg
Round1_T2D	0.7047	0.7975	0.86	—	—
Round4_2020	0.6133	—	0.91	0.985	0.92
2T_Round4	0.6130	—	0.89	0.810	0.54
HardTablesR2	0.5733	0.5610	0.93	—	0.91
HardTablesR3	0.3578	0.3367	0.62	—	0.90

Note: Alligator results are from Table III in [12] (PN + RN ranking with types). LinkingPark results are from Table 1 in [15]. SemTab Top-10 averages are from [25] and [28]. Empty cells indicate the result is not available for that system and dataset combination.

The gap between ReFinED and the purpose-built SemTab systems is substantial, which is expected given the fundamental difference in evaluation setting described in section 5.1.1. Alligator, which was trained on these exact datasets, achieves F1 scores of 0.86 on **Round1_T2D**, 0.91 on **Round4_2020**, and 0.93 on **HardTablesR2**, all well above ReFinED’s corresponding scores of 0.705, 0.613, and 0.573. This gap is smallest on **HardTablesR3**, where Alligator scores 0.62 and ReFinED scores 0.358, reflecting the difficulty of this dataset for all systems. On **Round4_2020**, LinkingPark achieves 0.985 compared to ReFinED’s 0.613, a gap of 0.372 F1 points. The Alligator scores in table 5.6 represent automated pre-HITL performance, with human review capable of pushing these scores even higher, as described in section 2.4.

The most informative comparison, however, is on **2T_Round4**. This dataset was specifically designed to challenge disambiguation systems with high-ambiguity homonyms, synthetic typos, and misspelt mentions, conditions that particularly expose the weakness of string-matching and index-based approaches. Here, ReFinED achieves 0.613, exceeding the SemTab 2020 top-10 average of 0.54 [25] and notably approaching LinkingPark’s 0.810 and Alligator’s 0.89 [15] [12]. This convergence is significant, as it suggests that ReFinED’s

transformer-based embeddings provide genuine robustness to lexical noise by resolving ambiguous mentions through semantic similarity rather than exact string matching. This interpretation is further supported by observations in Alligator, where Elasticsearch-based retrieval alone scored only 0.62 on `2T_Round4` [12], identical to ReFinED’s score, while the addition of neural re-ranking improved this to 0.89. This implies that the baseline retrieval difficulty on adversarial data is similar for both systems, but Alligator’s supervised re-ranking recovers performance that ReFinED’s zero-shot approach cannot.

A qualitative comparison with Belotti et al. [17] further contextualises these results. Their evaluation of Alligator, DAGOBAN, TURL, TableLlama, GPT-4o, and GPT-4o-mini operates under a closed-world assumption where the correct entity is always present in the candidate set, making direct numerical comparison with ReFinED’s open-world F1 scores invalid. However, their results demonstrate that even purpose-built and fine-tuned systems struggle on out-of-domain data, with accuracy dropping significantly when moving from in-domain to moderately out-of-domain settings. This mirrors the pattern observed in ReFinED’s own results, where performance on the adversarial HardTables datasets falls well below that on structured web tables, suggesting that dataset difficulty rather than system architecture is the primary driver of performance variation across the benchmark.

5.4 Qualitative Error Analysis

Beyond aggregate metrics, understanding why ReFinED fails on specific inputs is important for characterising the practical boundaries of neural entity linking on tabular data. This section presents a qualitative analysis of recurring failure patterns, identified through a systematic extraction of per-prediction details across four target datasets: `HTR1`, `companies`, `movies`, and `Round1_T2D`¹. For each dataset, 200 predictions were randomly sampled across both cell and row inference modes, giving 1600 predictions in total. Three recurring failure categories emerged from this analysis:

Lexical Ambiguity and Confidence Misprediction. The most prominent failure mode is high-confidence misprediction, where ReFinED commits confidently to an incorrect entity. These cases are particularly problematic because the model’s internal confidence score provides no warning signal, the prediction appears reliable but is wrong. A clear example of this is the mention “*Antarctic Treaty area*”, where the model predicted Q182814 (Antarctic Treaty System) with a confidence of 0.9797, when the correct entity was Q21590062 (Antarctic Treaty area). The two entities are semantically related and share overlapping surface forms, causing the model to favour the more prominent and frequently occurring entity, a direct consequence of the global entity prior $\hat{P}(e|m)$ in the ω scoring mechanism, which biases predictions toward popular entities. A total of 45

¹Cases were identified using a custom error analysis script logging prediction QIDs, ground truth QIDs, and confidence scores. Link to full script and output logs are available in Appendix A.

such high-confidence wrong predictions (confidence ≥ 0.9 , *is_correct* = 0) were identified across the sampled datasets, underscoring that high confidence is not a reliable indicator of correctness when semantically related candidates compete.

Semantic Drift in Row-Context Mode. A second failure mode specific to row-context inference is semantic drift, where neighbouring cells introduce misleading context that causes the model to override an otherwise correct cell-level prediction. This directly explains the row-mode performance degradation observed on adversarial datasets in section 5.2. A representative example is the mention “*Balfouria*”, correctly resolved to Q2479614 (Balfouria, plant) in cell mode, but incorrectly predicted as Q87179197 (Balfouria, settlement) when the row context included the value “435”. Similarly, “*copper-75*” was correctly linked to Q18844913 (copper-75) in cell mode, but the addition of numeric row values (“1.222 / -1 / -54471.341”) caused the model to predict Q2529274 (isotope of copper) instead. In both cases, the surrounding numeric or uninformative cells appear to distort the model’s contextual embedding, pulling the prediction toward an unrelated entity. Across the sampled datasets, 39 such semantic drift cases were identified where cell mode was correct but row mode was wrong for the same input.

Knowledge Base Coverage and NIL Failures. The most frequent failure mode by volume is *NIL* prediction on non-*NIL* ground truth entities, with 379 such cases identified across the sampled datasets. While some of these may reflect entities absent from ReFinED’s Wikidata snapshot due to the model’s knowledge cut-off, a more nuanced pattern emerges when examining cases where the model predicts *NIL* with maximum confidence (1.0). Examples include “*Abyssinian Ground Hornbill*” (ground truth Q480086) and “*Runddeln*” (ground truth Q7379764), both predicted as *NIL* with confidence 1.0. This apparent paradox, maximum confidence yet no entity predicted, can be explained by ReFinED’s scoring mechanism: a confidence of 1.0 in the normalised output indicates that the *NIL* candidate dominated the softmax distribution, meaning all real entity candidates scored negatively. This suggests these entities exist in Wikidata but are absent from ReFinED’s filtered entity set of approximately 33 million entries, representing a knowledge base coverage gap rather than confidence calibration failure. A further subset of *NIL* failures involves corrupted or noisy mentions such as “*Seeshaupt statio*” and “*lswout iatehouse*”, where character-level noise in the HTR datasets prevents candidate generation entirely, consistent with the adversarial design of those datasets described in section 3.1.3.

Summary. Taken together, these three failure categories reveal that ReFinED’s limitations on tabular data are not random but systematic. High-confidence mispredictions stem from entity prior bias toward popular entities. Semantic drift is an inherent risk of row-context inference on noisy tables. *NIL* failures on known entities largely reflect coverage gaps in the underlying entity set rather than model uncertainty. These observations

inform the practical deployment recommendations discussed in chapter 7.

5.5 Evaluation Summary

This evaluation was designed around four objectives established in section 3.2: high-volume efficiency, disambiguation under ambiguity, real-world robustness, and domain adaptation. The results address all four: high-volume efficiency is quantified in section 5.3.2, disambiguation under ambiguity and real-world robustness are examined across the adversarial and specialised datasets in section 5.3.1 and section 5.3.3, and domain adaptation is evaluated through the ablation study in section 5.2 and discussed further in chapter 6. Beyond these objectives, the results provide a complex picture across the three hypotheses defined in section 5.1.2.

The **Benchmark Hypothesis (RQ1)** is partially confirmed. ReFinED demonstrates competitive performance on adversarial disambiguation tasks, most notably on `2T_Round4`, where it exceeds the SemTab 2020 top-10 average and approaches purpose-built systems, suggesting that transformer-based embeddings offer real robustness to lexical noise. However, performance on the synthetic HardTables datasets remains substantially below that of trained systems, reflecting the inherent disadvantage of zero-shot inference against dataset-specifically trained competitors. Row-level context consistently improves disambiguation on structured, semantically rich datasets but degrades performance on adversarial data, confirming that contextual signals are only beneficial when surrounding cells are informative. A supplementary ablation further demonstrates that column header context improves F1 on 9 out of 11 datasets at less computational cost, representing a more practical alternative to full row-context inference in resource-constrained deployments.

The **Adaptation Hypothesis (RQ2)** is confirmed. Supervised fine-tuning on domain-specific tabular data gives substantial F1 improvements on in-domain datasets, with gains of up to +0.191 on `movies` in row mode. The out-of-domain present a more complex picture, with `Round1_T2D` showing significant row-mode degradation consistent with catastrophic forgetting [31], discussed further in chapter 6.

The **Efficiency Hypothesis (RQ3)** is confirmed. Row-context inference roughly halves average throughput and nearly doubles peak VRAM consumption relative to cell mode, with the impact most noticeable on wide tables such as `companies` where throughput drops by approximately 4.2 $\times$. The three largest datasets were infeasible to run in row mode on consumer-grade hardware, establishing a clear practical boundary for potential deployment.

Taken together, these results suggest that ReFinED occupies a viable but constrained

niche in the STI landscape, capable of competitive zero-shot performance on adversarial and domain-specific data, but limited by the absence of dataset-specific training and the computational demands of row-context inference at scale.

Reproducibility and Data Access

To support the findings presented in this chapter and ensure the transparency of the benchmarking process, logs has been saved to the project repository. This repository contains raw `.csv` execution logs, high-resolution Precision-Recall visualisations, and error analysis on the datasets used to categorise model failure modes. A detailed guide to the repository structure and file contents is provided in **Appendix A**.

Chapter 6

Domain Adaptation and Optimisation

Having established a baseline performance profile for ReFinED across 11 benchmarks in chapter 5, this chapter investigates whether supervised fine-tuning on domain-specific tabular data can improve those results, directly addressing section 1.3. The experiments focus on two domain-specific datasets, **companies** and **movies**, and evaluate five distinct training configurations ranging from small merged datasets to full single-domain models.

6.1 Technical Rationale for Fine-Tuning

The native training of ReFinED relies on Wikipedia, where the data consists of well-structured text and clearly defined entity mentions. Tabular data, however, frequently contain isolated cell values, abbreviations, and domain-specific terminology that diverge significantly from this pre-training distribution. This mismatch between pre-training and target domain is a well-documented challenge in NLP, and prior work has shown that domain-specific fine-tuning can significantly improve downstream performance over general-purpose baselines [30] [32]. This motivates exploring whether targeted adaptation of ReFinED’s entity-typing and disambiguation layers can close the gap between general-purpose performance and domain-specific accuracy.

Fine-tuning ReFinED requires reformatting tabular rows into the **Doc** and **Span** objects expected by the ReFinED training interface. Two custom dataset methods, `get_movie_docs` and `get_companies_docs`, were implemented in `dataset_factory.py` to handle this conversion, constructing **Doc** objects from CSV rows where the primary entity (film title or company name) is annotated as a gold-labelled **Span** with its corresponding Wikidata QID. The full row text is passed as document context, providing the model with the same

row-level signals used during inference.

An alternative approach to full fine-tuning is parameter-efficient adaptation, most notably Low-Rank Adaptation (LoRA) [33], which freezes the pre-trained model weights and injects small trainable rank-decomposition matrices into the transformer layers. This significantly reduces the number of trainable parameters and GPU memory requirements during training, making it particularly attractive for resource-constrained settings. However, since ReFinED’s fine-tuning interface does not natively expose LoRA-compatible hooks, standard full fine-tuning was used as a simpler and more directly supported approach. LoRA-based adaptation of ReFinED remains an interesting direction for future work, later discussed in chapter 8.

6.2 Fine-Tuning Methodology and Configuration

All fine-tuning experiments were executed on System B (RTX 4070 Ti) described in section 4.1, using batch size 128 throughout. Training data was sourced exclusively from the `companies` and `movies` datasets, with a 90/10 train/validation split applied across all configurations. The validation set was held out from training and used only for evaluation during the fine-tuning process.

Five distinct training configurations were evaluated to assess the trade-off between training data volume, domain specificity, and generalisation:

merged_10k:	5,000 rows from each domain (10,000 total). Trained in approximately 15 minutes.
merged_60k:	30,000 rows from each domain (60,000 total). Trained in approximately 1 hour 37 minutes.
merged_full:	All available rows from both domains ($67,500 \times 2 = 135,000$ total). Trained in approximately 3 hours 13 minutes.
companies_full:	All 67,500 rows from <code>companies</code> only. Trained in approximately 1 hour 34 minutes.
movies_full:	All 67,500 rows from <code>movies</code> only. Trained in approximately 1 hour 21 minutes.

The merged configurations were designed to investigate cross-domain stability and the prevention of catastrophic forgetting [31], while the single-domain configurations establish the upper bound of accuracy within an isolated domain. All models were fine-tuned using

the `wikipedia_model_with_numbers` base model with the `wikidata` entity set, consistent with the inference configuration used throughout chapter 5.

6.3 Evaluation Strategy

Fine-tuned models are evaluated using a subject-centric row accuracy metric implemented in `accuracy.py`. For each row in the test corpus, the metric isolates the primary entity (film title in `movies`, company name in `companies`) and compares the model’s highest confidence QID prediction for that cell against the Wikidata ground truth. A row is marked correct only if the QIDs match exactly. Alongside accuracy, precision and F1 are also reported. Note that in this evaluation setting, recall equals accuracy since there are no true negatives in entity linking, every target cell either has a correct prediction (TP) or does not (FN).

This metric is consistent with the evaluation approach used throughout chapter 5, where predictions are compared against Wikidata ground truth QIDs at the cell level. The distinction here is that the `movies` and `companies` datasets contain only a single annotated entity per row, the film title or company name respectively, meaning evaluation naturally focuses on whether the primary subject of each row is correctly resolved.

6.4 Results: The Impact of Domain Specialisation

Since each dataset contains a single annotated entity per row, recall equals accuracy in this evaluation setting. Precision and F1 are reported alongside recall for completeness.

Table 6.1 presents the subject-centric accuracy, precision, and F1 for all five fine-tuned configurations against the stock baseline, evaluated on the test splits of both `movies` and `companies`, with all results using row mode inference, consistent with the full-row context used during fine-tuning.

Table 6.1: Fine-tuning Results: F1-score, Precision (P), and Recall (R) (Row Mode)

Model	Companies			Movies		
	F1	P	R	F1	P	R
Baseline (stock)	0.842	0.919	0.776	0.756	0.836	0.691
merged_10k	0.859	0.964	0.775	0.925	0.968	0.886
merged_60k	0.867	0.980	0.778	0.934	0.979	0.892
merged_full	0.877	0.992	0.786	0.941	0.997	0.892
companies_full	0.878	0.989	0.790	0.522	0.820	0.383
movies_full	0.145	0.888	0.079	0.939	0.990	0.893

Note: $F1 = F1\text{-score}$, $P = Precision$, $R = Recall$. Bold values indicate the best per metric among fine-tuned models.

Sources: Baseline data derived from `experimental_results_wiki_2026-05-08.csv` (row mode). All other configurations derived from `fine_tuning_logs.md`. See Appendix A for full raw data and logs.

Metric Interpretation. Performance is reported using Precision (P), Recall (R), and F1-score. Since each row contains exactly one target entity, recall is mathematically equivalent to row-wise accuracy. F1 serves as the primary metric, as it penalises models that achieve high precision by excessively abstaining from difficult links.

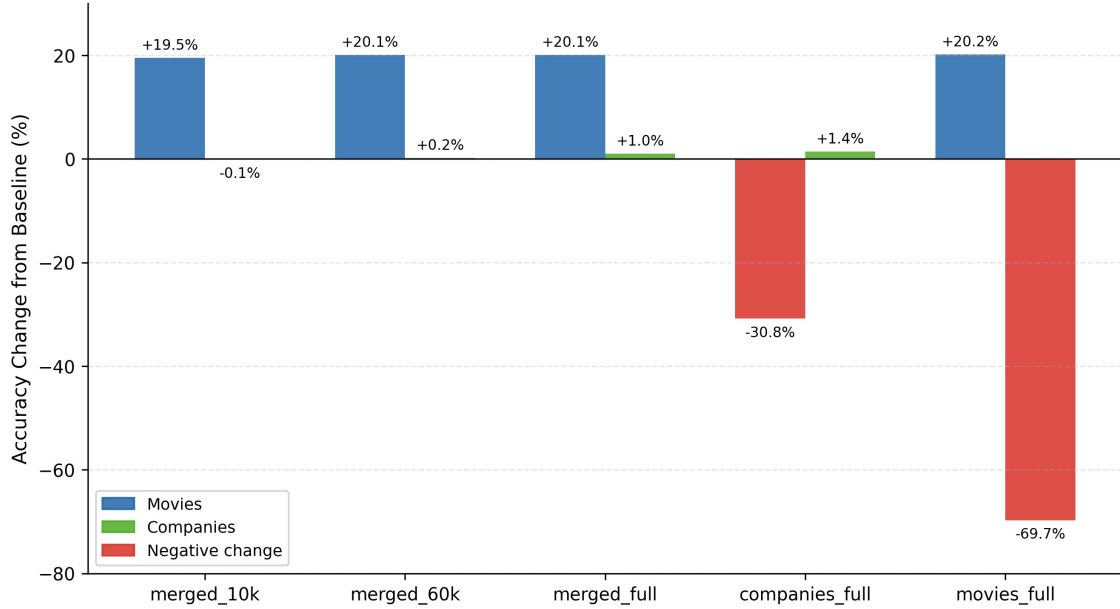


Figure 6.1: Accuracy change relative to the stock baseline for each fine-tuned model across both domains. Negative values indicate degradation relative to the stock model.

Three patterns emerge clearly from these results:

Merged models offer the best balance. Even the smallest merged configuration (`merged_10k`, trained in just 15 minutes) delivers substantial accuracy gains on `movies` (+19.5%), while `companies` accuracy remains largely stable across merged configurations, ranging from 77.5% to 78.6% against a baseline of 77.6%. This suggests that the stock model already performs well on corporate entity mentions in row mode, and that fine-tuning primarily benefits the more ambiguous movie title domain. The `merged_full` model achieves the best overall balance with 89.2% accuracy on `movies` and 78.6% on `companies`, near perfect precision of 0.997 and 0.992 respectively, and is used as the primary fine-tuned model for the ablation study in section 5.2.

Single-domain models show catastrophic forgetting. The single-domain configurations demonstrate the cost of over-specialisation. `movies_full` achieves the highest movies accuracy (89.3%) but collapses entirely on `companies`, dropping from 77.6% to just 7.9%, with F1 falling to 0.1451. Similarly, `companies_full` improves companies accuracy to 79.0% but causes severe degradation on `movies` (−30.8%, F1 = 0.522). This pattern is consistent with catastrophic forgetting [31], where fine-tuning on a narrow domain overwrites general-purpose weights, degrading performance on inputs outside the distribution. `movies_full` causes far greater cross-domain damage than `companies_full`, dropping companies accuracy by 69.7% compared to 30.8% drop in the reverse direction, likely because movie titles are more distinct and semantically distant from corporate entity mentions, causing a sharper shift in the model’s internal representations.

Diminishing returns beyond 60,000 rows. Comparing `merged_10k`, `merged_60k`, and `merged_full` reveals that accuracy gains plateau after approximately 60,000 training rows. On `movies`, the jump from 10k to 60k gives only an increase of +0.6%, and extending to 135k adds no further improvement. On `companies`, all merged configurations perform similarly, within 1.1% of each other, while training time triples from 1h 37m to 3h 13m for the full dataset. This suggests that a 60k merged dataset represents a practical sweet spot for domain adaptation on consumer-grade hardware, sufficient for near-maximum performance gains without excessive computational cost.

6.5 Summary and Implications

The fine-tuning experiments across five configurations further confirm the **Adaptation Hypothesis (RQ2)** from section 5.1.2, building on the ablation results in section 5.5 with stronger evidence. Supervised fine-tuning consistently improves in-domain performance while introducing measurable out-of-domain degradation. This pattern of results also carries practical implications for deployment.

The most important finding is that merged training data is strongly preferable to single-domain training for any deployment scenario that must serve multiple table types. The `merged_full` model achieves near identical accuracy to the single-domain specialists on their respective domains, while avoiding the catastrophic collapse observed in `movies_full` and `companies_full`. This suggests that the model benefits from seeing a diverse entity distribution during fine-tuning, which helps preserve the general linking capability of the base model while adapting its priors toward the target domains.

The diminishing returns observed beyond 60k training rows have direct practical implications: effective domain adaptation does not require large annotated corpora. A relatively small dataset of around 60,000 rows, realistically achievable in many real-world annotation scenarios, is sufficient to capture the majority of domain-specific performance gains. This is consistent with the findings from Gururangan et al. [30], who show that domain-adaptive pre-training gives most of its benefit from the initial exposure to in-domain text, with diminishing returns as data volume increases.

The generalisation results also raise an important consideration for practical deployment: a model fine-tuned on a narrow domain should not be used as a general-purpose linker without careful evaluation of its out-of-domain behaviour. As demonstrated by `movies_full`'s collapse on `companies`, the same fine-tuning that improves in-domain accuracy can dramatically degrade performance on structurally different entity types. This motivates the modular expert architecture discussed in chapter 7, where domain-specific models are deployed selectively based on incoming table metadata rather than as a universal replacement for the stock model.

Chapter 7

Discussion

The empirical results presented in chapter 5 and chapter 6 provide a detailed picture of ReFinED’s capabilities and constraints across a diverse set of tabular benchmarks. This chapter steps back from individual results and asks a broader question: what do these findings collectively reveal about the viability of a general-purpose zero-shot neural approach to tabular entity linking, and what does this mean for how such a system should be deployed in practice? The following sections examine ReFinED’s strengths and weaknesses as a system, reflect on what the dataset-specific results reveal about the boundaries of neural entity linking, and draw practical implications for deployment in real-world annotation pipelines.

7.1 Strengths and Limitations of ReFinED

ReFinED’s most significant strength is its zero-shot entity linking capability. Unlike purpose-built systems like Alligator, LinkingPark, and the broader SemTab field, evaluated in section 5.3.3, ReFinED requires no dataset-specific training, candidate index construction, or domain-tuned scoring functions. It enters every benchmark cold, relying entirely on its pre-trained RoBERTa backbone and Wikidata entity descriptions to resolve mentions through semantic similarity. This zero-shot capability is particularly valuable in open-domain annotation scenarios where labelled training data is unavailable or where the target entity distribution is not known in advance. The ReFinED paper reports inference speeds over 60 times faster than comparable zero-shot capable systems [11], and the benchmark results shown in table 5.5 in section 5.3.2 confirm that cell mode inference achieves an average throughput of 238.3 txt/s on consumer-grade hardware, making large-scale evaluation of millions of cells easily managed within hours rather than days.

A second strength is ReFinED’s robustness to lexical noise, inconsistencies in wording in adversarial data. On **2T_Round4**, a dataset deliberately constructed with homonyms, synthetic typos, and misspelled mentions, ReFinED achieves a CEA F1 of 0.613, exceeding the SemTab 2020 top-10 average of 0.54 [25] and matching Alligator’s Elasticsearch-based retrieval score of 0.62 [12] before its neural re-ranking stage. As discussed in section 5.1.1, this alignment suggests that transformer-based embeddings provide genuine robustness to surface-form variation, as the model resolves ambiguous mentions through semantic similarity rather than exact string matching, which is precisely the condition where index-based retrieval struggles most.

The open-world *NIL* prediction mechanism is a third notable strength. ReFinED includes a special *NIL* candidate with a fixed score of zero in every prediction, meaning it abstains from linking when no candidate scores positively. This is critical for tabular annotation tasks where entities may be absent from the knowledge base, and stands in contrast to closed-world systems such as TURL and TableLlama, which always assign an entity regardless of confidence. This capability is particularly relevant for **2T_Round4** (0.5% explicit *NIL*) and **SN** (22.9% *NIL*) [29], the two datasets in this evaluation that contain genuine ground truth *NIL* entries, as well as for real-world deployment scenarios where out-of-knowledge-base entities are common.

These strengths are accompanied by clear limitations. The most significant is the performance gap relative to purpose-built systems on synthesised HardTables benchmarks. In **HardTablesR2** and **HardTablesR3**, ReFinED scores 0.573 and 0.358, respectively, against SemTab top-10 averages of 0.91 and 0.90 [28]. This gap is largely structural. The competing systems were trained on SemTab data, tuned against its specific distributions, and in Alligator’s case benefit from human-in-the-loop revision. ReFinED, by design, does not receive any of these advantages. The experiments in chapter 6 demonstrate that fine-tuning on in-domain data can significantly close this gap for specific domains: F1 improvements of up to +0.191 on **movies**, but also introduce the risk of catastrophic forgetting on out-of-domain datasets, most severely seen by the **movies_full** model’s collapse on **companies**.

A second limitation is the conservative *NIL*-prediction tendency that consistently suppresses recall across all evaluated datasets. As established in section 5.3.1, ReFinED’s precision systematically exceeds recall because the model defaults to *NIL* whenever no candidate scores positively in the ω scoring mechanism. While this is correct behaviour for open-world evaluation, it means the model misses a significant proportion of linkable entities, a pattern reflected in the precision-recall curves showing optimal thresholds around 0.3 – 0.4 for SemTab datasets. The gap between these peak values and the F1 scores achieved at default settings suggests that the model operates more conservatively than optimal for recall, and that threshold tuning could recover additional true posit-

ives without substantially sacrificing precision. This underlines a common challenge in modern deep learning: as architectures increase in complexity, they often become poorly calibrated, which means that confidence scores assigned to predictions do not accurately reflect the probability of correctness [34].

A third limitation is the knowledge base coverage constraint. ReFinED’s Wikidata entity set contains approximately 33 million filtered entities, a large but finite subset of Wikidata’s full catalogue. As observed in section 5.4, entities that exist in Wikidata but fall outside this filtered set are predicted as *NIL* with maximum confidence, a silent failure mode that is indistinguishable from real *NIL* predictions in evaluation logs. Combined with the model’s 2022 knowledge cut-off, this creates a coverage gap that is particularly relevant for dynamic domains such as companies, where new entities are continuously being added to Wikidata after the model’s training snapshot.

7.2 Dataset-Specific Observations

The results show clear patterns across different types of datasets that are not obvious from just looking at overall scores. These patterns seem to come from both the nature of the datasets and how ReFinED is built. Together, they give a clearer picture of where neural entity linking works well on tables, and where it tends to struggle.

Structured web tables favour row-context inference. Structured web tables benefit significantly from row-context inference when surrounding cells contain more informative semantic signals. In such cases, neighbouring values help disambiguate entity mentions more effectively. For instance, `Round1_T2D` and `movies` show the largest improvements (+0.093 and +0.179 F1), likely because these tables are well-structured and include columns such as director, year, or country, which provides strong contextual cues. In contrast, `companies` and `SN` show smaller gains (+0.039 and +0.018 F1), consistent with their shorter and more abbreviated row structures, where neighbouring cells contribute less semantic information. Overall, this suggests that row-context inference is most effective when tables contain rich and meaningful relationships between columns.

Adversarial datasets expose architectural boundaries. The `HardTables` and `HTR` datasets are a different kind of challenge altogether. On `HardTablesR2` and `HardTablesR3`, using row context actually makes things worse, with drops of -0.012 and -0.021 F1. As discussed in section 5.4, this seems to come from semantic drift, where noisy or unhelpful neighbouring cells end up pulling the model toward the wrong entities instead of helping. `HardTablesR3` stands out as the hardest dataset overall, with an F1 of just 0.358 in cell mode and 0.337 in row mode. Its single-column structure, combined with deliberately ambiguous or adversarial naming, leaves very little useful context to work with. In practice, this exposes a clear limitation of ReFinED’s disambiguation when both surface form and

context are unreliable.

The 2T_Round4 dataset is the most informative comparison point. Among all evaluated datasets, 2T_Round4 stands out as the most useful for comparing ReFinED with purpose-built systems. Its adversarial setup, with highly ambiguous names and lexical noise, directly targets the weaknesses of string matching and index-based methods. This is where ReFinED’s transformer-based representations provide a clear advantage. ReFinED achieves an F1 score of 0.613, which is higher than the SemTab 2020 top-10 average of 0.54 [25] on this dataset. This suggests that zero-shot neural linking can outperform the field average in exactly the kinds of cases where heuristic methods tend to break down. There is still a noticeable gap to systems like LinkingPark (0.810) and Alligator (0.890), which benefit from supervised re-ranking and additional candidate refinement. At the same time, the overall difficulty of the dataset is similar, since Alligator’s retrieval-only score of 0.62 is very close to ReFinED’s zero-shot performance.

NIL calibration and coverage challenges. Round4_2020 and SN both represent one of the largest evaluation challenges in terms of volume and domain specificity. In Round4_2020, which contains nearly one million linkable targets, the model achieves a cell mode F1 of 0.6133, but shows a heavy skew toward precision (0.7867) over recall (0.5025). SN shows a similar pattern, with precision reaching 0.8074 while recall remains low at 0.4150. This suggests that on large-scale or specialised datasets, ReFinED defaults to a conservative stance, preferring to abstain via a *NIL* prediction rather than risk a false positive. On SN, this is further compounded by genuine ground-truth *NIL* entries and gaps in KB coverage for specialised procurement entities. Ultimately, both datasets confirm that while the model is highly reliable when it commits to a link, its primary failure mode is "silent omission", where it fails to resolve entities due to high ambiguity or missing representation in the filtered Wikidata snapshot.

SN and 2T_Round4 are the only datasets with genuine ground truth *NIL* entries, comprising 22.9% [29] and 0.5% [26] of annotations respectively. On SN, performance is further complicated by the nature of the data, as procurement tables often contain abbreviated and inconsistently formatted mentions, and many entities have limited or no representation in Wikidata. The F1 score remains around 0.59 regardless of the confidence threshold as shown in section 5.3.1, suggesting that performance is mainly limited by missing coverage rather than poor calibration.

7.3 Implications for Practical Deployment

The benchmark and fine-tuning results carry concrete implications for how ReFinED should be deployed in production annotation pipelines. Three considerations stand out: the computational trade-off between inference modes, the architecture of a multi-domain

deployment, and the role of confidence scores in supporting human review workflows.

7.3.1 Inference Mode Trade-offs

The efficiency results in section 5.3.2 establish an apparent trade-off between accuracy and throughput when choosing between cell and row mode inference. Switching to row mode roughly halves average throughput from 238.3 to 120.1 txt/s and nearly doubles peak VRAM consumption from 1.91 to 3.51 GB on average. The impact is most severe on wide tables, where `companies` drops from 117.1 to 28.1 txt/s, a 4.2 $\times$ reduction. While narrow, sparse tables like `HardTablesR3` show high throughput in row mode (228.2 txt/s) due to minimal concatenation overhead.

For practical deployment this suggests a tiered inference strategy. Cell mode should be the default for large-scale or time-sensitive annotation tasks, as it processes millions of cells within hours on consumer-grade hardware. Row mode should be reserved selectively for smaller, semantically rich tables where disambiguation gains justify the computational cost, most notably datasets similar to `movies` and `Round1_T2D`, where row context provides F1 improvements of +0.179 and +0.093 respectively. An intermediate option, column header context (Configuration D in section 5.2), achieves the highest average F1 score (0.6086) at a significantly lower computational overhead than row-context inference. The three largest datasets in this evaluation, `Round3_2019`, `Round4_2020`, and `2T_Round4`, were impractical in row mode on consumer hardware entirely, confirming that row mode is not viable for web-scale annotation without significantly more powerful infrastructure. This trade-off directly informed the design of the Koala-UI back-end described in section 4.4, where cell mode is used as the default inference strategy.

7.3.2 Domain Expert Architecture

The fine-tuning results in chapter 6 show that a single universal model is not well suited for multi-domain deployment. Fine-tuning on a narrow domain can severely hurt performance on other domains, as seen with `movies_full` dropping `companies` accuracy by 69.7 percentage points due to catastrophic forgetting. The `merged_full` model handles this better through diverse training data, but still falls short of single-domain models with their specific domains.

A more practical approach would be a modular setup, where domain-specific fine-tuned models are used alongside the stock model rather than replacing it. Incoming tables could be routed to the right model based on simple signals like column headers or dataset identifiers. New domains can be added as separate modules without retraining the entire model, which keeps long-term maintenance manageable. The stock model would still handle unknown domains through its zero-shot capability, while the specialist models take over when domain-specific training data is available. The `merged_full` model is a good

starting point for this, covering both `movies` and `companies` with near-perfect precision of 0.997 and 0.992 respectively in row mode.

7.3.3 Human-in-the-Loop Utility

ReFinED’s confidence score $\omega \in [0, 1]$ can be useful for guiding human review in annotation workflows. As shown in section 5.4, a high confidence score does not always mean the prediction is correct. 45 high-confidence wrong predictions (confidence ≥ 0.9) were found across the sampled datasets, where the model committed to the wrong entity with no visible sign of uncertainty. These cases are hard to catch because simple confidence thresholding would not flag them.

This is similar to the problem that motivated Alligator’s human-in-the-loop design [12], where confidence scores are used to rank predictions and direct user attention toward the most uncertain links first. Alligator’s experiments show that reviewing just 10 to 30% of predictions, starting from the least confident, is enough to reach F1 scores above 0.98 on most datasets. A similar approach could work well with ReFinED, where users focus on low-confidence predictions while trusting the high-confidence ones. The main limitation is that some wrong predictions come with high confidence due to entity prior bias toward popular entities, as described in section 5.4. These would need additional checks to detect, such as flagging cases where multiple candidates score similarly.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This thesis set out to evaluate ReFinED as a general-purpose zero-shot entity linking tool for tabular data, assess whether its performance could be improved through methods such as fine-tuning, and demonstrate its practical applicability through integration with the Koala-UI annotation platform. Across these three objectives, the work produced a comprehensive empirical picture of where neural entity linking currently stands relative to purpose-built STI systems, and under what conditions a zero-shot approach can be considered a viable alternative.

The benchmark evaluation across 11 datasets, spanning SemTab benchmarks, adversarial benchmarks, and domain-specific corpora, makes up the primary contribution of this thesis. The results show that ReFinED performs well on adversarial disambiguation tasks where lexical noise and surface-form variation are the main challenge, achieving a CEA F1 of 0.613 on 2T_Round4 and exceeding the SemTab 2020 top-10 average on that dataset. On synthesised benchmarks like HardTables, it falls behind purpose-built systems that were trained specifically on SemTab data. This is expected given how ReFinED works. It enters every benchmark without any prior exposure to the dataset, relying entirely on its pre-trained RoBERTa backbone and Wikidata entity descriptions. The fact that it still performs competitively on adversarial data suggests that transformer-based semantic similarity is a genuinely robust approach to disambiguation when string-matching methods fail.

The row-context inference experiments add an interesting point of view to this. Row context consistently improves performance on well-structured, semantically rich tables, with gains of up to +0.179 F1 on *movies*, but hurts performance on adversarial datasets

where noisy neighbouring cells introduce misleading context rather than helpful signals. This shows that row-context inference is not always better and should be used selectively depending on the table’s structure.

A supplementary finding from the column header context experiments further demonstrates that prepending column labels to cell values improves F1 on most of the used datasets at less cost, suggesting that structural metadata is a practical and under-explored source of disambiguation signal for tabular entity linking.

The fine-tuning experiments, while not the main focus, produced a useful practical finding: a merged dataset of around 60,000 rows is enough to capture most of the domain-specific performance gains in our test scenario, with the `merged_full` model reaching near-perfect precision of 0.997 on `movies` and 0.992 on `companies` in row mode. The catastrophic forgetting seen in single-domain models shows how important training data diversity is, and supports the modular expert architecture discussed in chapter 7 as a more practical deployment strategy.

The Koala-UI integration was intended as a technical demonstration to view the results of ReFinED’s entity linking visually, rather than a core research contribution. It shows that ReFinED can be wrapped as a lightweight REST API that fits into existing annotation systems, and that its confidence scores give a natural way to prioritise links for human review. In this context, Koala-UI serves as an effective visual demonstration platform, leveraging an existing open-source annotation system with built-in human-in-the-loop capabilities to illustrate how ReFinED can support interactive and assisted entity linking workflows.

Overall, a zero-shot ReFinED is best seen not as a replacement for purpose-built STI systems, but as a useful complement for situations where labelled training data is not available, where entity distributions are unknown, or where the ability to abstain from linking is just as important as the ability to link correctly. For anyone working with diverse or fast-changing tabular data, a zero-shot neural approach like this is a practical and increasingly capable starting point that can be further strengthened through targeted fine-tuning.

8.2 Future Work

Several potential directions for future work emerge naturally from the limitations and findings of this thesis. Together, these point to clear opportunities to improve both the robustness and overall performance of entity linking in tabular data.

Fine-Tuning on SemTab-Scale data. The fine-tuning experiments in this thesis were conducted on a small set of domain-specific datasets, namely `companies` and `movies`, due to computational constraints. A natural extension would be to investigate fine-tuning ReFinED directly on larger SemTab benchmark data, following an approach similar to TURL [16], which was pre-trained on large-scale Wikipedia tables with table-specific objectives. Training on the full SemTab corpus would expose the model to a significantly broader range of table structures, entity types, and disambiguation challenges, potentially improving generalisation across all evaluated datasets. Of particular interest would be whether exposure to the adversarial mechanisms present in HardTables, such as character-level noise and synthetic typos, during fine-tuning could improve robustness on those datasets, where the stock model performed weakest. Such training would likely require server-grade GPU resources and careful curation of training data to preserve the model’s general-purpose linking capability.

Domain Routing and Multi-Expert Architecture. A further direction for future research is the development of an automated domain router, a lightweight component that analyses incoming table metadata such as header strings or column type predictions to automatically select the most appropriate fine-tuned expert model from a curated library of domain-specific weights. This builds directly on the modular expert concept discussed in section 7.3.2, where the performance gains observed for `companies` and `movies` suggest that specialised models consistently outperform the general-purpose baseline on in-domain data. Scaling this approach across broader domains such as medical, legal, or financial tables would require maintaining multiple model checkpoints, introducing storage and deployment overhead. Parameter-efficient adaptation methods such as LoRA, discussed in section 6.1, are therefore particularly relevant, since they reduce each domain-specific model to a small set of low-rank weight matrices rather than a full model checkpoint, significantly reducing the storage and deployment cost of maintaining a library of expert models. Combined with the asynchronous job-based API architecture developed in this thesis, such a system could form the foundation of a practical and scalable entity linking service capable of serving diverse table types without sacrificing domain-specific accuracy.

Efficiency via Parameter-Efficient Adaptation. The fine-tuning experiments in this thesis used full model fine-tuning, which requires storing a complete copy of the model weights for each domain. As discussed in section 6.1, LoRA [33] offers a more resource-efficient alternative, injecting small trainable rank-decomposition matrices into the transformer layers while keeping the base weights frozen. This would allow multiple domain-specific adaptations to coexist with a much smaller memory footprint, making the modular expert approach practically feasible on consumer-grade hardware. Integrating LoRA support into ReFinED’s fine-tuning interface would require modifications to the training pipeline but represents a straightforward engineering extension with potentially

significant practical benefits.

LLM-Assisted Verification. A promising direction for improving annotation quality is using large language models as a secondary verification step for low-confidence ReFinED predictions. Rather than replacing ReFinED’s linking mechanism, an LLM such as GPT-4o or a locally hosted model like Llama could be queried to validate or correct predictions where ReFinED’s confidence score falls below a threshold. This hybrid approach would combine ReFinED’s speed and scalability for bulk annotation with the broader world knowledge and reasoning capabilities of LLMs for handling ambiguous or uncertain cases. Belotti et al. [17] demonstrate that LLM-based approaches achieve strong performance on STI benchmarks, and a verification-only role would sidestep their main limitations of closed-world assumption and high per-sample cost. Initial experiments could target the high-confidence wrong predictions identified in section 5.4, where entity prior bias causes overconfident errors that simple thresholding cannot detect.

Guided Disambiguation via External Candidate Injection. While this thesis evaluates ReFinED as an end-to-end system, a promising extension would be to investigate its performance when constrained by external candidate lists. ReFinED’s architecture inherently supports passing pre-defined candidate entities for specific spans. Evaluating ReFinED under these conditions would allow for a more granular assessment of its disambiguation layers by neutralising potential errors in mention detection and initial candidate generation. This is particularly relevant for benchmarks like HTR1 and HTR2, which provide fixed `mention_to_qid` mappings, and would provide a direct comparison of ReFinED’s re-ranking capabilities against more traditional, search-based STI models.

Bibliography

- [1] A. Gandomi and M. Haider, “Beyond the hype: Big data concepts, methods, and analytics,” *International Journal of Information Management*, vol. 35, no. 2, pp. 137–144, 2015.
- [2] T. R. Gruber, “A translation approach to portable ontology specifications,” *Knowledge Acquisition*, vol. 5, no. 2, pp. 199–220, 1993.
- [3] A. Hogan, E. Blomqvist, M. Cochez, C. D’amato, G. D. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann, “Knowledge graphs,” *ACM Computing Surveys*, vol. 54, p. 1–37, July 2021.
- [4] R. Reiter, “On closed world data bases,” in *Readings in Artificial Intelligence* (B. L. Webber and N. J. Nilsson, eds.), pp. 119–140, Morgan Kaufmann, 1981.
- [5] G. Limaye, S. Sarawagi, and S. Chakrabarti, “Annotating and searching web tables using entities, types and relationships,” *Proceedings of the VLDB Endowment*, vol. 3, no. 1-2, pp. 1338–1347, 2010.
- [6] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, “A neural probabilistic language model,” *Journal of machine learning research*, vol. 3, no. Feb, pp. 1137–1155, 2003.
- [7] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems* (I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, eds.), vol. 30, Curran Associates, Inc., 2017.

- [9] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” 2019.
- [10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bi-directional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (J. Burstein, C. Doran, and T. Solorio, eds.), (Minneapolis, Minnesota), pp. 4171–4186, Association for Computational Linguistics, June 2019.
- [11] T. Ayoola, S. Tyagi, J. Fisher, C. Christodoulopoulos, and A. Pierleoni, “Refined: An efficient zero-shot-capable approach to end-to-end entity linking,” 2022.
- [12] R. Avogadro, M. Ciavotta, F. De Paoli, M. Palmonari, and D. Roman, “Estimating link confidence for human-in-the-loop table annotation,” in *2023 IEEE/WIC International Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT)*, pp. 142–149, 2023.
- [13] R. Avogadro, M. Cremaschi, F. D’Adda, F. De Paoli, M. Palmonari, *et al.*, “Lamapi: a comprehensive tool for string-based entity retrieval with type-base filters,” in *OM@ ISWC*, pp. 25–36, 2022.
- [14] V.-P. Huynh, J. Liu, Y. Chabot, F. Deuzé, T. Labbé, P. Monnin, and R. Troncy, “DAGOBAB: Table and Graph Contexts for Efficient Semantic Annotation of Tabular Data,” in *CEUR Workshop Proceedings*, vol. 3103 of *Proceedings of the Semantic Web Challenge on Tabular Data to Knowledge Graph Matching co-located with the 20th International Semantic Web Conference (ISWC 2021)*, (En ligne, Unknown Region), p. 2, Oct. 2021.
- [15] S. Chen, A. Karaoglu, C. Negreanu, T. Ma, J.-G. Yao, J. Williams, A. Gordon, and C.-Y. Lin, “Linkingpark: An integrated approach for semantic table interpretation,” *SemTab@ ISWC*, vol. 2020, 2020.
- [16] X. Deng, H. Sun, A. Lees, Y. Wu, and C. Yu, “Turl: Table understanding through representation learning,” *SIGMOD Rec.*, vol. 51, p. 33–40, June 2022.
- [17] F. Belotti, F. Dadda, M. Cremaschi, R. Avogadro, and M. Palmonari, “Evaluating LLMs on entity disambiguation in tables,” *arXiv preprint arXiv:2408.06423*, 2024.
- [18] P. Nguyen, I. Yamada, N. Kertkeidkachorn, R. Ichise, and H. Takeda, “Mtab4wikidata

- at semtab 2020: Tabular data annotation with wikidata.,” *SemTab@ ISWC*, vol. 2775, pp. 86–95, 2020.
- [19] N. Abdelmageed and S. Schindler, “Jentab: Matching tabular data to knowledge graphs.,” in *SemTab@ ISWC*, pp. 40–49, 2020.
 - [20] Z. Zhang, “Effective and efficient semantic table interpretation using tableminer+,” *Semantic Web*, vol. 8, no. 6, pp. 921–957, 2017.
 - [21] T. Zhang, X. Yue, Y. Li, and H. Sun, “TableLlama: Towards open large generalist models for tables,” in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (K. Duh, H. Gomez, and S. Bethard, eds.), (Mexico City, Mexico), pp. 6024–6044, Association for Computational Linguistics, June 2024.
 - [22] E. Jiménez-Ruiz, O. Hassanzadeh, V. Efthymiou, J. Chen, and K. Srinivas, “Semtab 2019: Resources to benchmark tabular data to knowledge graph matching systems,” in *The Semantic Web* (A. Harth, S. Kirrane, A.-C. Ngonga Ngomo, H. Paulheim, A. Rula, A. L. Gentile, P. Haase, and M. Cochez, eds.), (Cham), pp. 514–530, Springer International Publishing, 2020.
 - [23] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.
 - [24] D. Ritze, O. Lehmberg, and C. Bizer, “Matching html tables to dbpedia,” in *Proceedings of the 5th International Conference on Web Intelligence, Mining and Semantics, WIMS ’15*, (New York, NY, USA), Association for Computing Machinery, 2015.
 - [25] E. Jimenez-Ruiz, O. Hassanzadeh, V. Efthymiou, J. Chen, K. Srinivas, and V. Cutrona, “Results of semtab 2020,” *CEUR Workshop Proceedings*, vol. 2775, pp. 1–8, January 2020. Copyright © 2020 for this paper by its authors.
 - [26] V. Cutrona, F. Bianchi, E. Jiménez-Ruiz, and M. Palmonari, “Tough tables: Carefully evaluating entity linking for tabular data,” in *The Semantic Web – ISWC 2020* (J. Z. Pan, V. Tamma, C. d’Amato, K. Janowicz, B. Fu, A. Polleres, O. Seneviratne, and L. Kagal, eds.), (Cham), pp. 328–343, Springer International Publishing, 2020.
 - [27] R. Avogadro, “Benchmark datasets for entity linking from tabular data,” July 2025.
 - [28] V. Cutrona, J. Chen, V. Efthymiou, O. Hassanzadeh, E. Jimenez-Ruiz, J. Sequeda, K. Srinivas, N. Abdelmageed, M. Hulsebos, D. Oliveira, and C. Pesquita, “Results

of semtab 2021,” in *20th International Semantic Web Conference*, vol. 3103, pp. 1–12, CEUR Workshop Proceedings, March 2022. Copyright © 2021 for the individual papers by the papers’ authors. This volume and its papers are published under the Creative Commons License Attribution 4.0 International (CC BY 4.0).

- [29] R. Avogadro, I. Makgill, A. Thomas, A. Soylu, and D. Roman, “Building a Canonical Register of Public Sector Entities: Semantic Linking of Procurement Data at Scale,” in *Joint Proceedings of Industry, Doctoral Consortium, Posters and Demos of the 24th International Semantic Web Conference (ISWC-C 2025)*, vol. 4085 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2025.
- [30] S. Gururangan, A. Marasović, S. Swayamdipta, K. Lo, I. Beltagy, D. Downey, and N. A. Smith, “Don’t stop pretraining: Adapt language models to domains and tasks,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, eds.), (Online), pp. 8342–8360, Association for Computational Linguistics, July 2020.
- [31] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, “Overcoming catastrophic forgetting in neural networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [32] J. Howard and S. Ruder, “Universal language model fine-tuning for text classification,” in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (I. Gurevych and Y. Miyao, eds.), (Melbourne, Australia), pp. 328–339, Association for Computational Linguistics, July 2018.
- [33] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” 2021.
- [34] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proceedings of the 34th International Conference on Machine Learning* (D. Precup and Y. W. Teh, eds.), vol. 70 of *Proceedings of Machine Learning Research*, pp. 1321–1330, PMLR, 06–11 Aug 2017.

Appendix A

Experimental Logs and Benchmark Data

All raw experimental results, visualisation plots, and high-resolution logs generated during this research are available in the project’s data repository. This data provides the evidence for the findings presented in chapters 5 to 7.

A.1 Data Repository Access

The complete dataset, including fine-tuning summaries and error analysis files, can be accessed at the following URL:

https://github.com/enRichMyData/ReFinED/tree/main/my_tests/logs,
https://github.com/enRichMyData/ReFinED/tree/main/my_tests/analysis

A.2 Repository Structure and Contents

Fine-Tuning and Optimization Data

`fine_tuning_logs.md` A markdown log containing training parameters and detailed per-model metrics (P , R , $F1$) for the five experimental models.

`fine_tuning_summary.pdf` Summary of the early-stage training runs and coarse-grained accuracy checks. (Same data as `fine_tune_logs.md`).

`experimental_results_f1_0.8972_2026-03-07.csv` Raw performance data for the configuration achieving peak multi-domain balance (`merged_full`).

Benchmark Evaluation Logs

experimental_results_wiki_2026-05-08.csv Full raw evaluation data for all Wikidata-based benchmarks.

experimental_results_wiki_2026-05-08_summary.csv A human-readable summary of the primary benchmark metrics across all datasets.

error_analysis.csv The data used for the qualitative error analysis in section 5.4, including cases of entity prior bias and context-driven failures.

Visualisation and Confidence Data

confidence_scores_wiki_2026-05-08.csv Individual confidence scores used to generate the precision-recall curves.

confidence_semtab_cell.png / row.png High-resolution plots of confidence scores across the SemTab datasets.

confidence_specialised_cell.png / row.png High-resolution plots for the specialised movies, companies, and SN datasets.

Appendix B

Dataset Repository References

Note on Dataset Naming and Consistency: The datasets listed below are structurally identical to the official SemTab corpora. Minor dataset name variations and marginal differences in local file counts compared to official reports result from the removal of unmapped or malformed source files during the local audit process to ensure experimental integrity.

Table B.1: Zenodo Record IDs and DOI Mappings for Evaluated Corpora.

Dataset Group / Suite	Zenodo ID	Source DOI / URL
<i>SemTab Challenge Rounds</i>		
Round 1 (T2D)	3518539	https://zenodo.org/records/3518539
Round 3 (2019)	3518539	https://zenodo.org/records/3518539
Round 4 (2020)	4282879	https://zenodo.org/records/4282879
HardTables R2/R3	6154708	https://zenodo.org/records/6154708
<i>EnRichMyData Benchmark Suite</i>	15888942	https://zenodo.org/records/15888942
HTR1 & HTR2		
Movies		
Companies		
Spend Network (SN)		

Appendix C

API Endpoint Specifications

This appendix documents the REST API endpoints exposed by the ReFinED FastAPI service described in section 4.4. The service follows the provider interface pattern used by Koala-UI, allowing it to be registered and used alongside other reconciliation providers such as Crocodile and LionLinker.

Table C.1: ReFinED API Endpoint Summary

Method	Endpoint	Description
GET	/	Health check and service status.
POST	/link	Synchronous entity linking for single strings.
POST	/jobs	Submits a new reconciliation job (inline or multipart).
GET	/jobs/{id}	Returns job status (queued, running, done, failed).
GET	/jobs/{id}/results	Retrieves paginated results for a completed job.

The service supports both synchronous queries via `/link` and asynchronous batch processing. For tabular data, a client submits rows to the `/jobs` endpoint, receiving a unique job identifier. Progress is tracked via polling, and once complete, results are retrieved in a Koala-compatible JSON structure. Each result entry contains the input mention, the predicted Wikidata QID, a human-readable entity label, and a normalised confidence score $\omega \in [0, 1]$.

Appendix D

Source Code Snippets

The following snippet shows the core accuracy measurement function used throughout the evaluation pipeline, implemented in `accuracy.py`. *NIL* predictions are treated as valid true positives when the ground truth is also *NIL*, reflecting the open-world evaluation setting.

The complete implementation is available in the project repository¹.

¹<https://github.com/enRichMyData/ReFinED>

Listing D.1: Subject-centric accuracy measurement (accuracy.py)

```
def measure_accuracy(pred_spans, truths, verbose=False):
    total, tp, fn, fp = 0, 0, 0, 0

    for pred_span, truth_qids in zip(pred_spans, truths):
        if not truth_qids:
            continue
        total += 1

        pred_qid = (
            getattr(pred_span[0].predicted_entity,
                    "wikidata_entity_id", None)
            if pred_span else None
        )
        pred_qid = pred_qid if pred_qid is not None else "NIL"

        is_match = (pred_qid in truth_qids or
                    (pred_qid == "NIL" and "NIL" in truth_qids))

        if is_match:
            tp += 1
        else:
            fn += 1

        if pred_qid != "NIL" and pred_qid not in truth_qids:
            fp += 1

    accuracy = tp / (total + 1e-8)
    precision = tp / (tp + fp + 1e-8)
    recall = tp / (tp + fn + 1e-8)
    f1 = 2 * (precision * recall) / (precision + recall + 1e-8)

    return EvalMetrics(
        accuracy, precision, recall,
        f1, tp, fp, fn
    )
```